

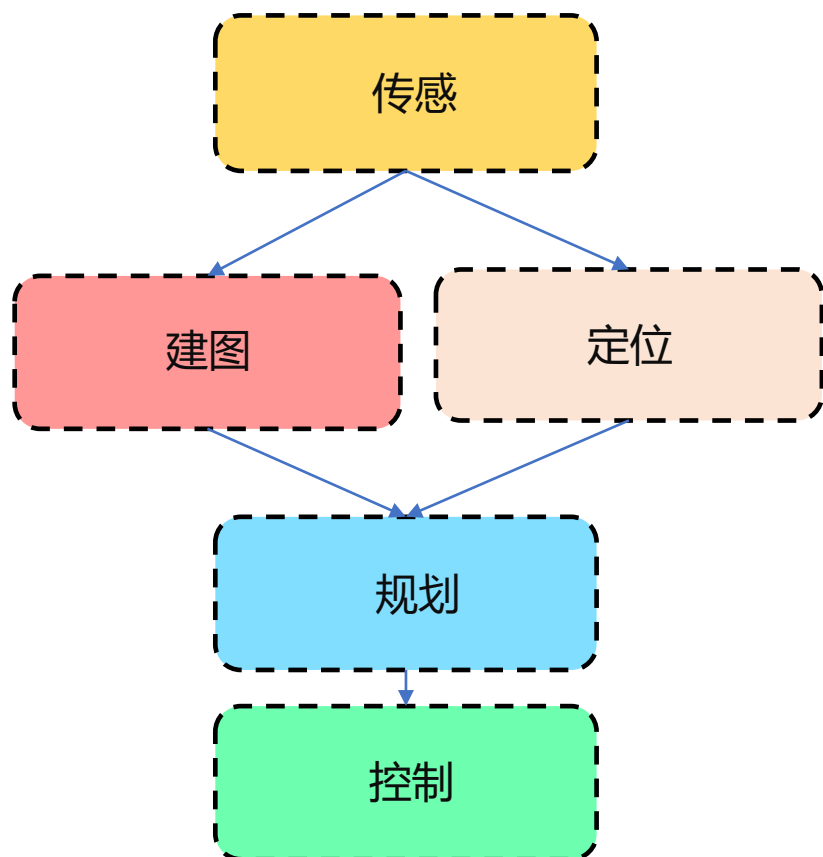
无人机路径规划算法

课程回放及答疑：

<https://www.shenlanxueyuan.com/my/course/385>

运动规划基本概念

- 感知-规划-控制：闭环系统



- 基本要求

- 安全性：躲避障碍物
- 光滑性：节省能量、提升运动连续性
- 动力学可行性：可执行、可控制

- 经典链路

- 前端：路径搜索
 - 搜索初始可行路径
 - 低维度问题
 - 离散空间
- 后端：轨迹生成
 - 搜索机器人可执行轨迹
 - 高维度问题
 - 连续空间

通用导航系统架构

● 估计

- 低延迟
- 高精度 & 一致性

● 感知

- 三维 & 稠密感知
- 地图维护 & 作用于规划



● 规划

- 应对复杂 & 未知环境
- 安全 & 动力学可行性
- 有限的传感器 & 计算能力

● 控制

- 高机动运动
- 光滑轨迹跟踪

- 定位模块：根据传感器实时输入，实现对机器人自身姿态的轨迹，反馈控制模块。定位模块要求精度、一致性、实时性及恶劣环境下的鲁棒性。
- 建图模块：根据传感器信息，实时构建环境三维几何模型，建立地图用于规划中的碰撞检测。建图模块要求地图稠密度、细粒度及建图实时性。
- 规划模块：以机器人实时位姿估计为轨迹起始状态，导航目标为终止状态，生成符合动力学模型，且保证安全性的运动轨迹。规划模块要求生成轨迹的安全性、光滑性、动力学可行性。



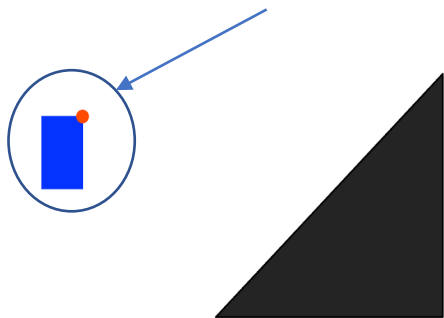
构形空间

- 机器人构形: 机器人所有的位置点
- 机器人自由度(DOF): 用来表示机器人构形的最小实数坐标数
- 机器人构形空间: 包含所有可能的机器人构形的 n 维空间, 记作C-space
- 每个机器人姿态都是C-Space中的一个点

构形空间障碍物

- 在工作空间中规划
 - 机器人具有不同的形状和大小
 - 碰撞检测需要知道机器人的几何信息——费时，难度大

检查机器人形状是否碰撞



(1) 矩形移动机器人

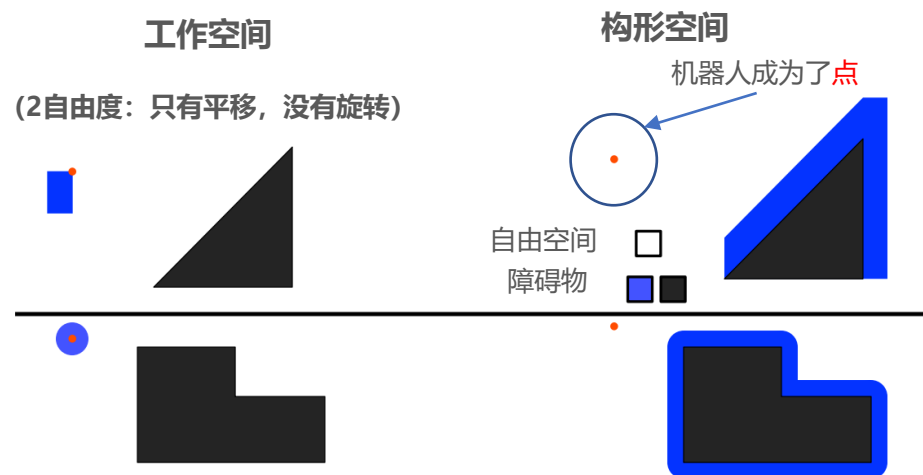


(2) 圆形移动机器人

构形空间障碍物

- 在构形空间中规划

- 机器人表示为C-space中的一个点，如位置 (R^3 中的点)，姿态 ($SO(3)$ 中的点)
- 障碍物需要在构形空间中表示 (先于运动规划完成)，称为构形空间障碍，或C-obstacle
- $C\text{-space} = (C\text{-obstacle}) \cup (C\text{-free})$
- 路径规划就是在C-free中寻找起点 q_{start} 至终点 q_{goal} 的路径



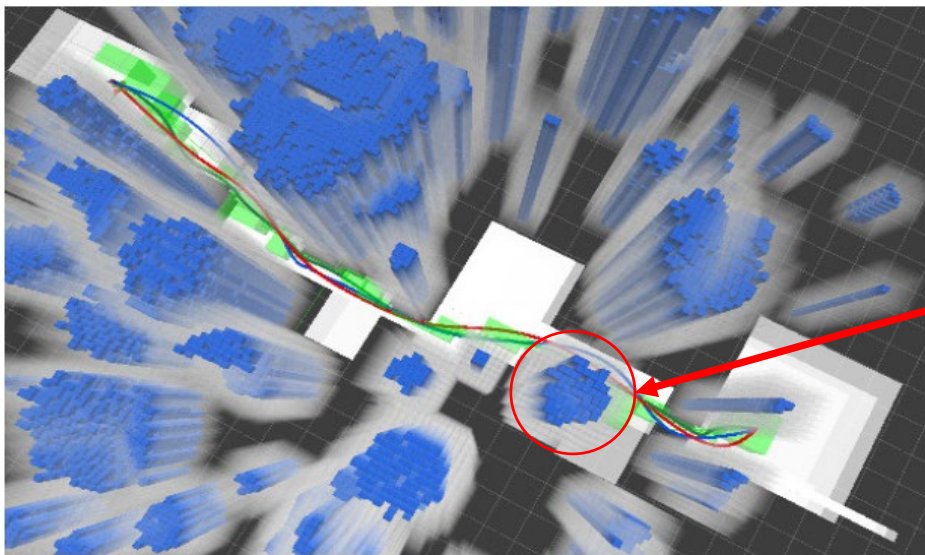
工作空间和构形空间障碍物

- 工作空间

- 机器人有形状和大小（不利于运动规划）

- 构形空间： C-space

- 机器人是一个点（便于运动规划）
 - 运动规划前，障碍物先在C-space中表示
- 在C-space中表示障碍物非常复杂。在实际中使用近似（但是更保守）的表示方法



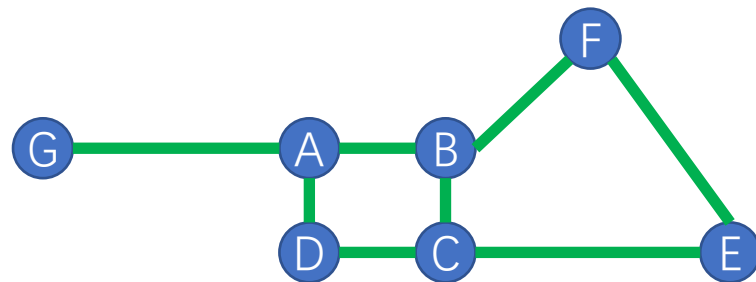
如果我们保守地把机器人建模成一个半径为 δ_r 的球体，构建C-space可以把所有障碍物向各个方向膨胀 δ_r 。

图的定义

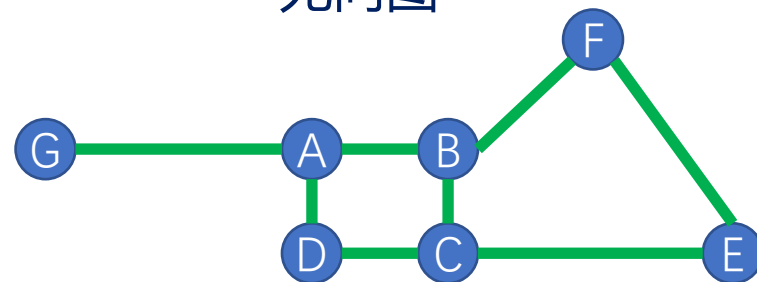
图 (Graphs)

图由节点和边构成

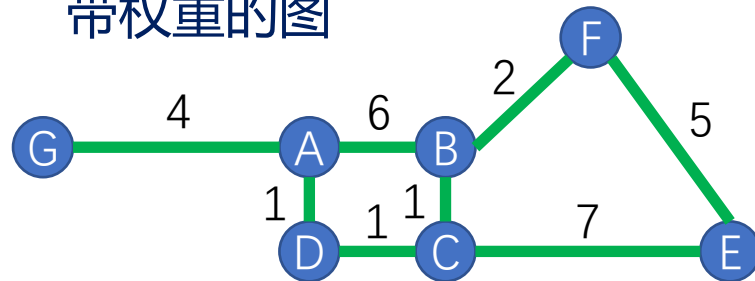
不带权重的图



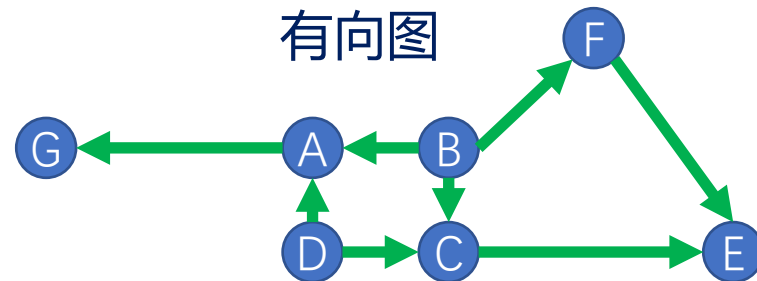
无向图



带权重的图

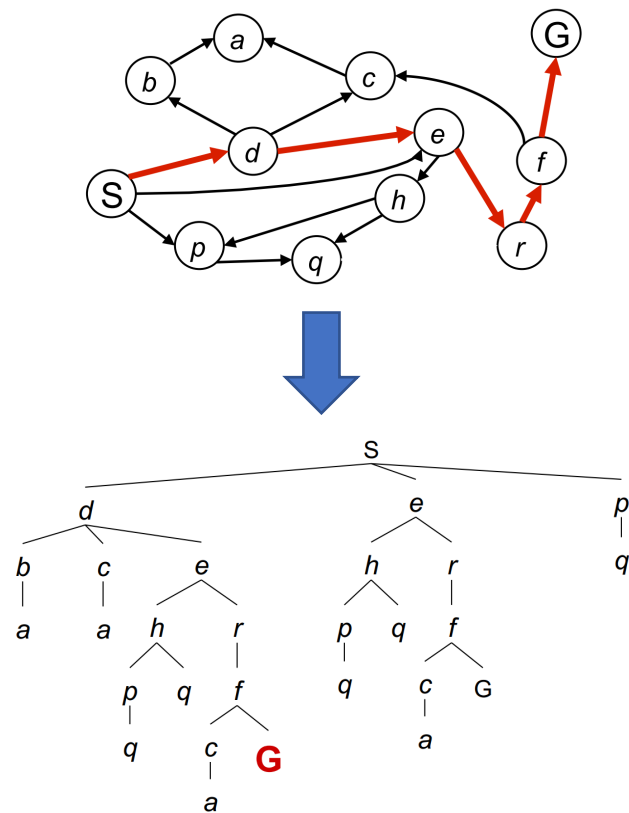


有向图



图搜索算法概述

- 搜索过程开始于初始点S
 - 搜索过程会生成一个搜索树
 - 对树中节点逆向搜索可以得到一条路径
 - 很多情况下，构建图的整棵搜索树是不明智的（工作量大且低效）——尽快到达目标点才是我们的目的



图搜索算法概述

- 维护一个容器来存储所有待访问的节点
- 容器初始化时只存在初始状态 X_s
- 开始循环
 - 删除Remove: 根据指定的规则从容器中移出一个节点
 - 扩展Expansion: 得到该节点的所有邻节点
 - 存入Push: 将这些邻节点存入容器
- 结束循环

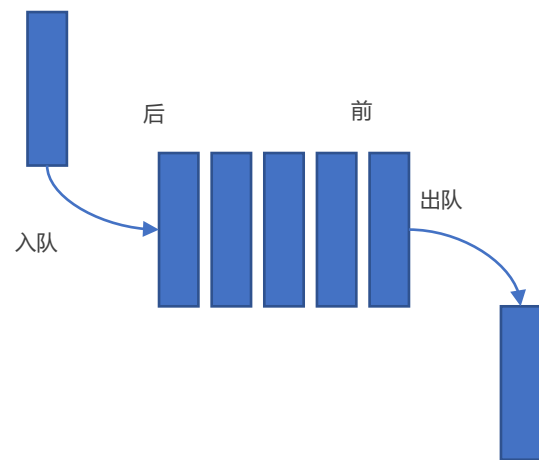


图搜索基础

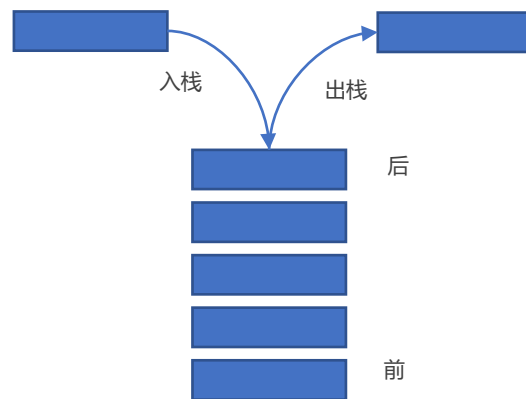
- Question 1: 如何退出循环?
 - 例: 容器空掉时退出循环
- Question 2: 如果图中存在回环?
 - 禁止从容器中移出的节点再次进入容器
- Question 3: 如何制定取出节点的规则使 **目标尽快被达到**，且尽可能少的扩展结点？

图的遍历

- 广度优先搜索（BFS）与 深度优先搜索（DFS）



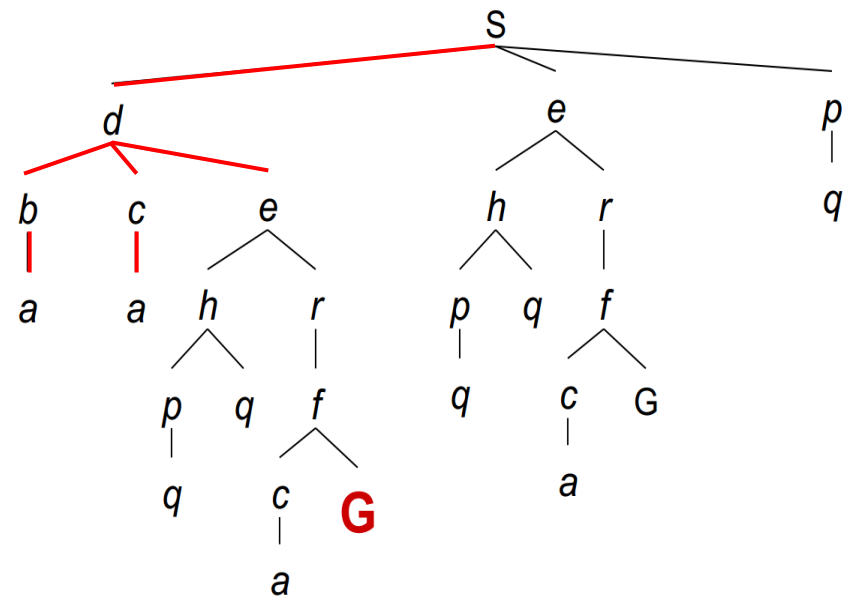
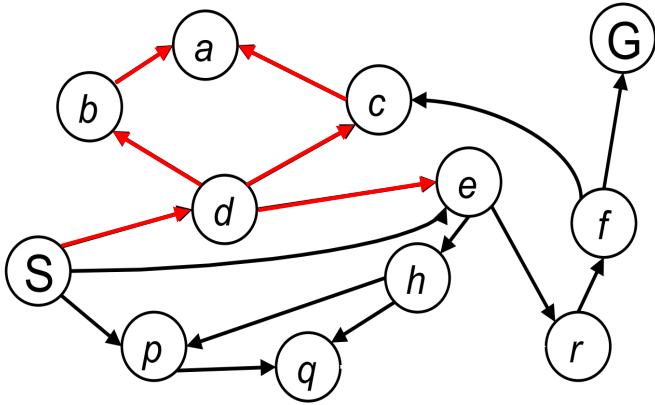
队列



堆栈

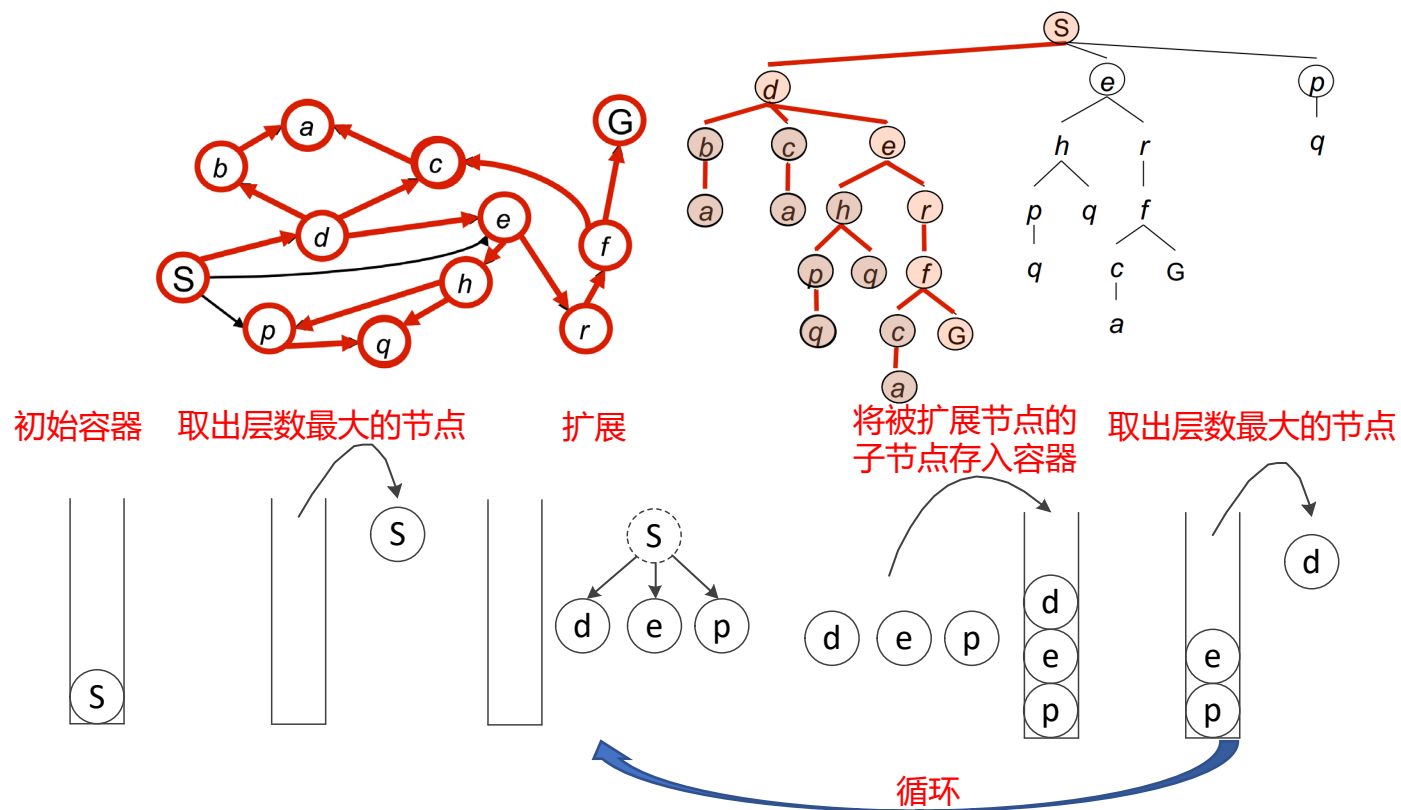
深度优先搜索

- 策略：每次取出容器中最深的节点



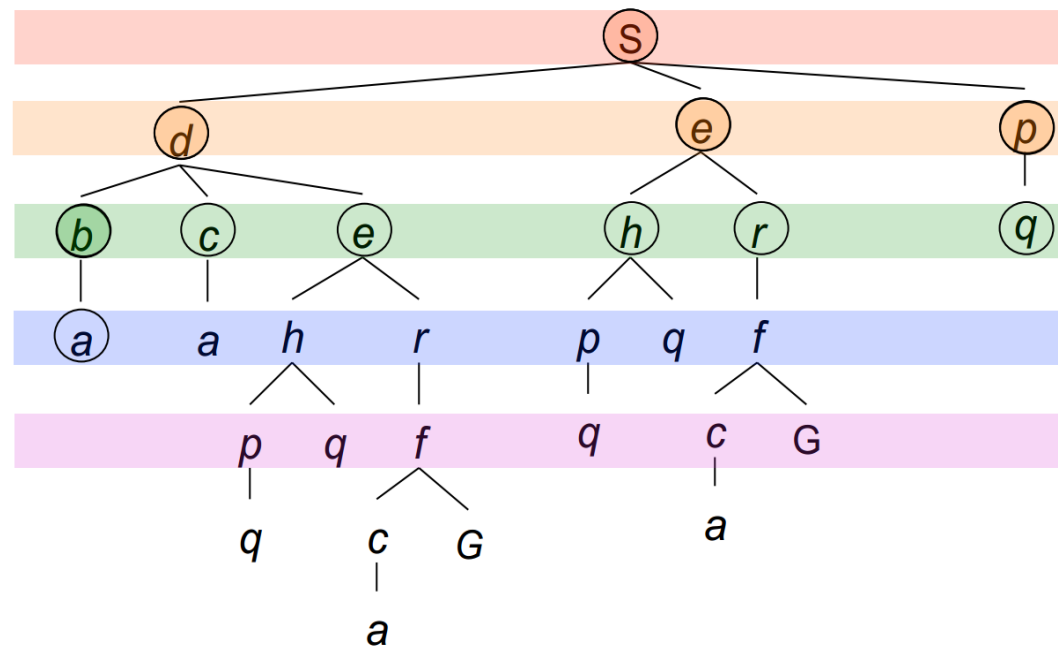
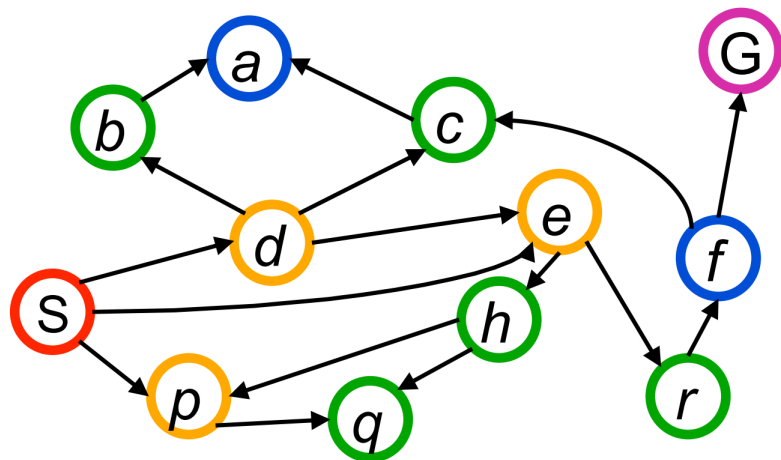
深度优先搜索

- 维护一个后进先出（last in first out, LIFO）的容器，即堆栈



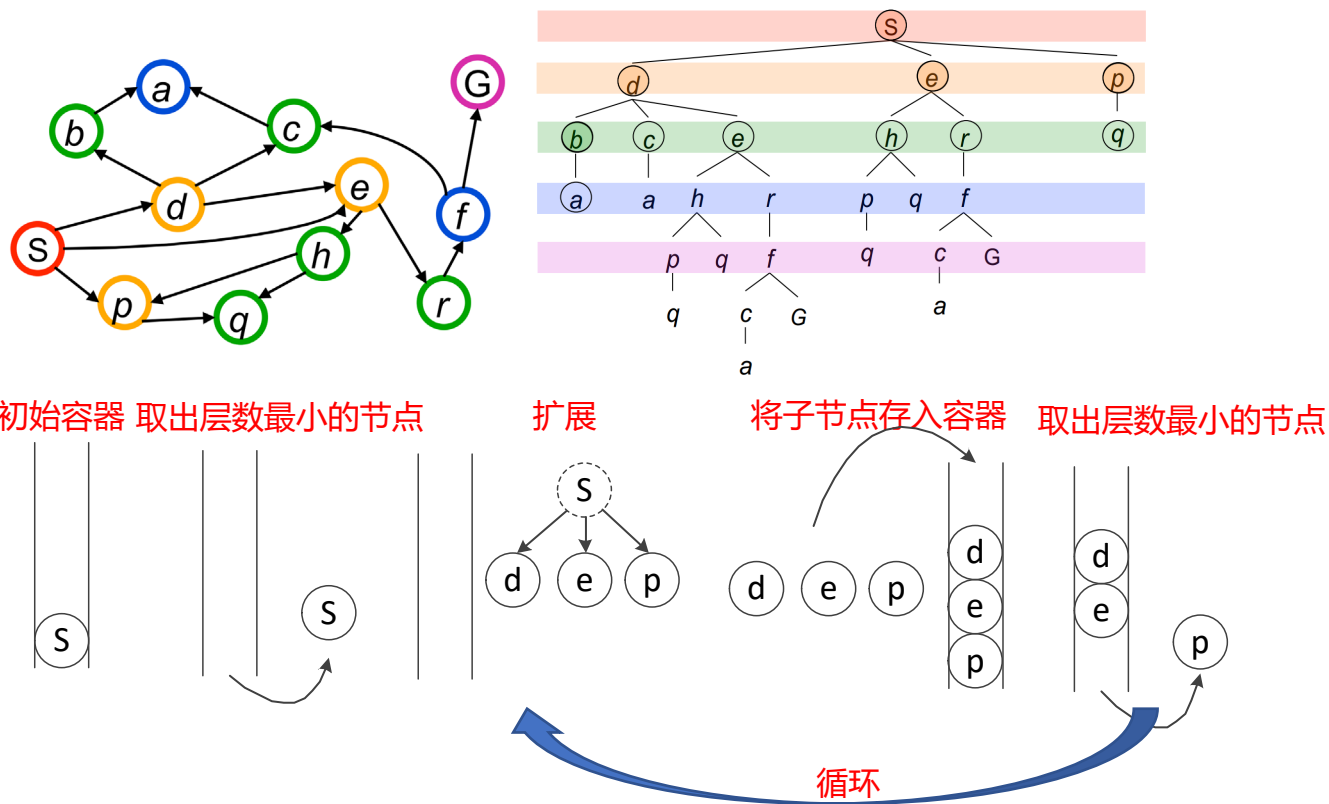
广度优先搜索

- 策略：每次取出容器中最浅的节点



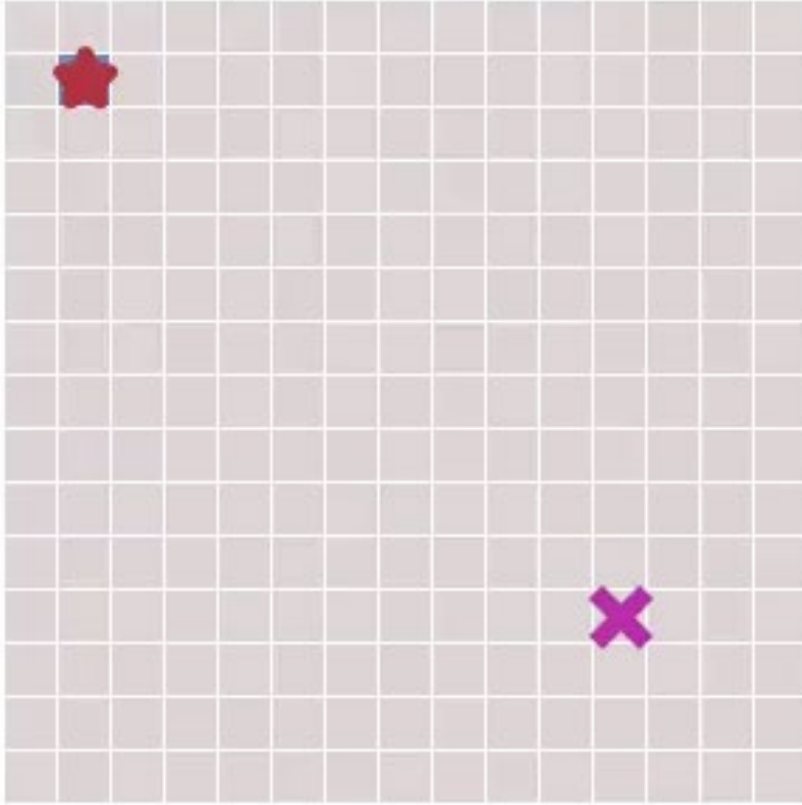
广度优先搜索

- 维护一个先进先出（first in first out, FIFO）的容器，即队列

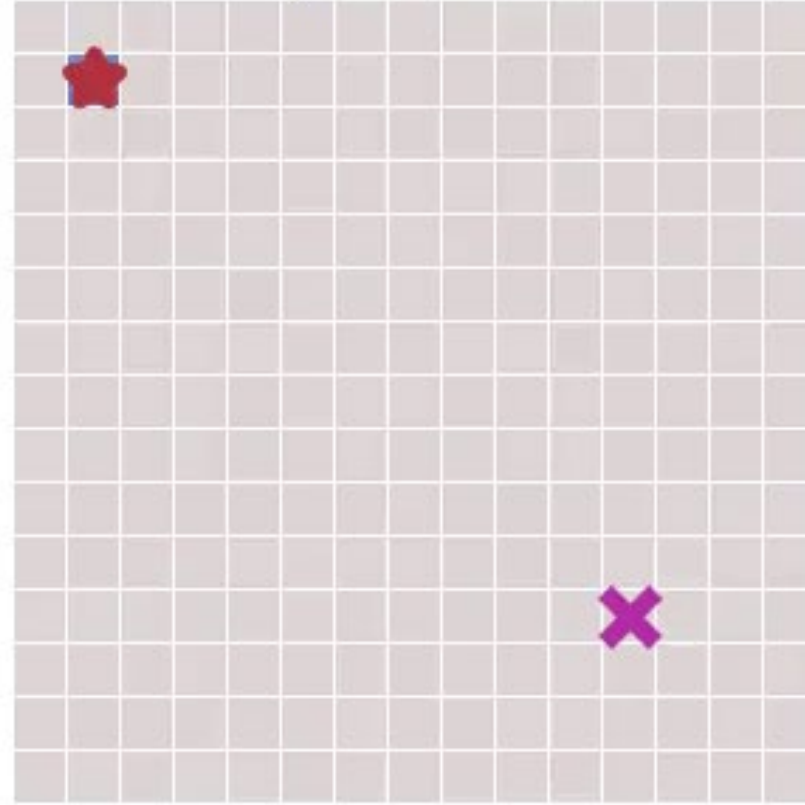


BFS vs. DFS

Breadth First Search



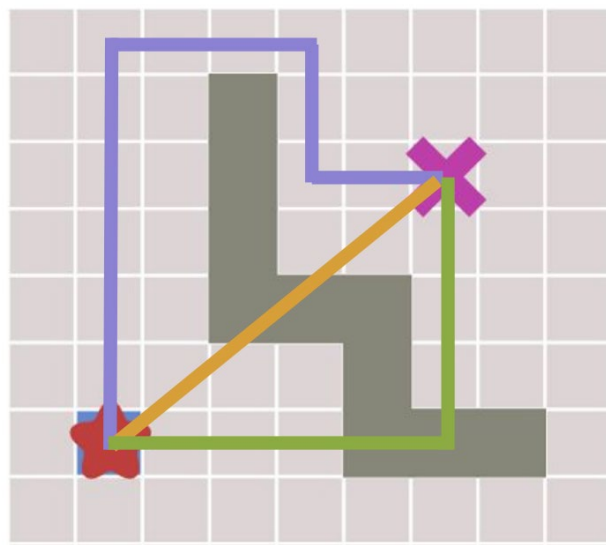
Depth First Search



请记住BFS

贪婪最佳优先算法

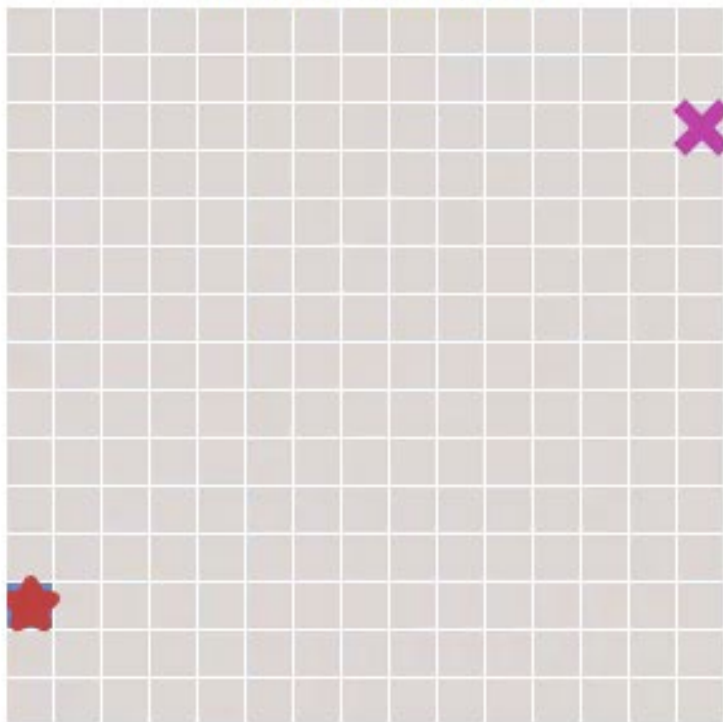
- BFS和DFS取出节点的策略基于访问的先后顺序
- 贪婪最优算法通过启发式函数取出节点
- 定义：启发式函数是猜测距离目标有多近
 - 引导搜索方向
 - 易于计算



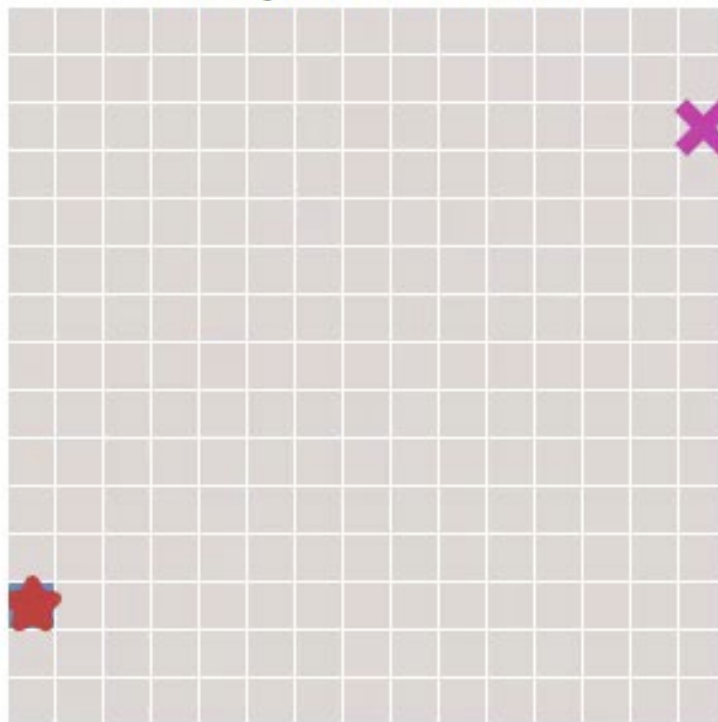
- 欧式距离
- 曼哈顿距离

贪婪最佳优先算法

Breadth First Search



Greedy Best-First Search

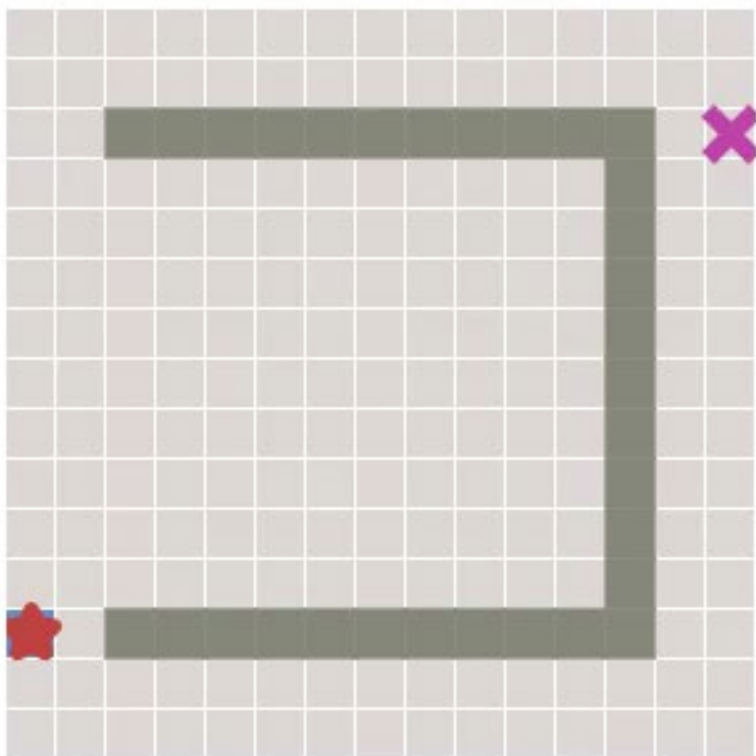


看起来不错~

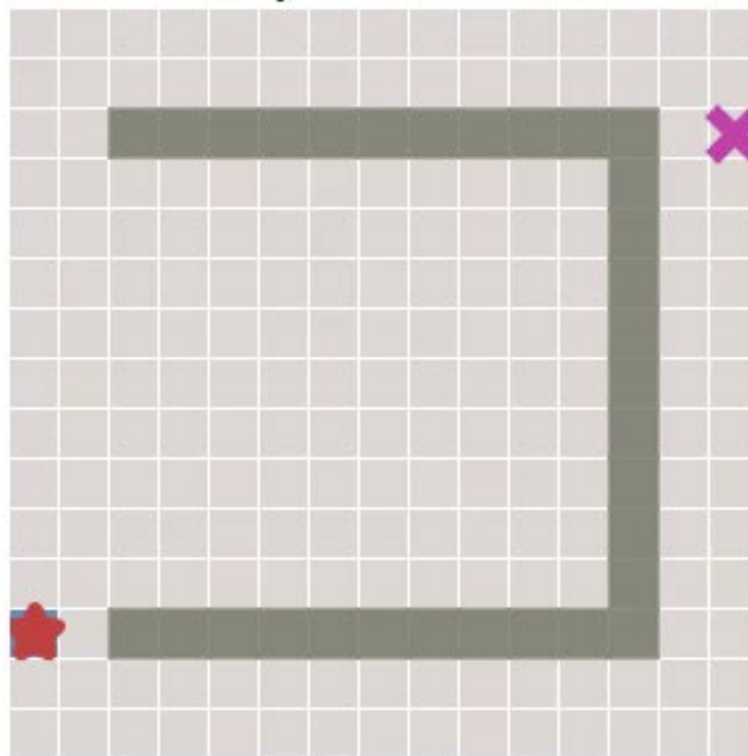
Courtesy: Amit Patel's Introduction to A*, Stanford

贪婪最佳优先算法

Breadth First Search



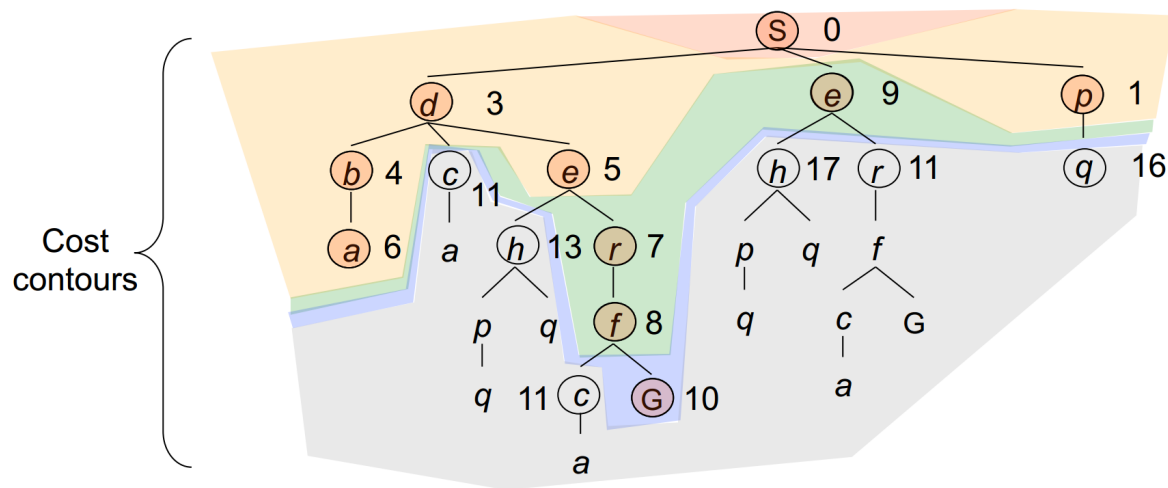
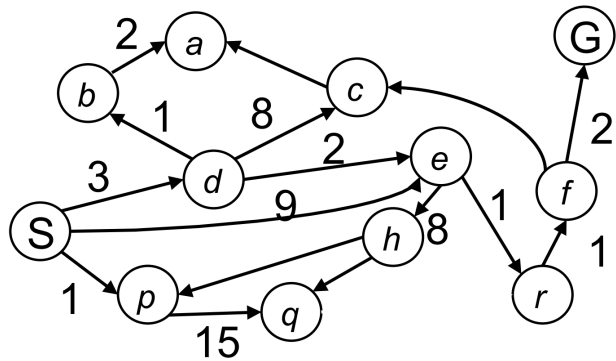
Greedy Best-First Search



可是当出现障碍物时...

Dijkstra 算法

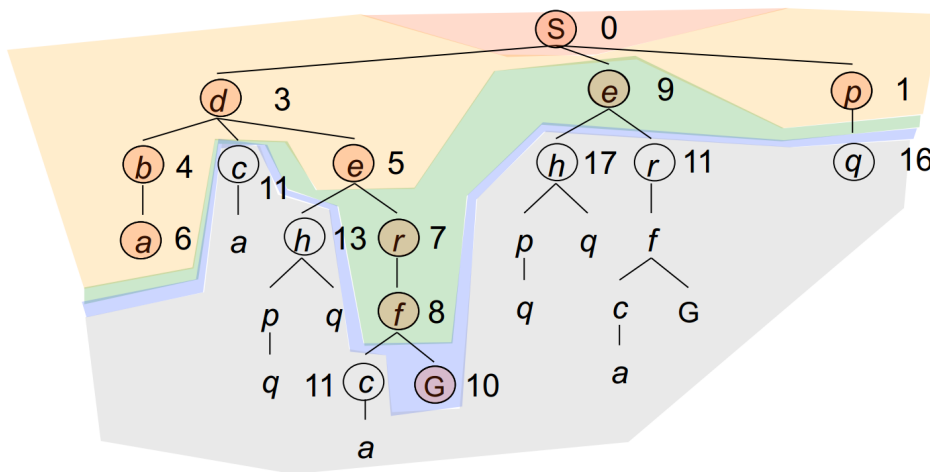
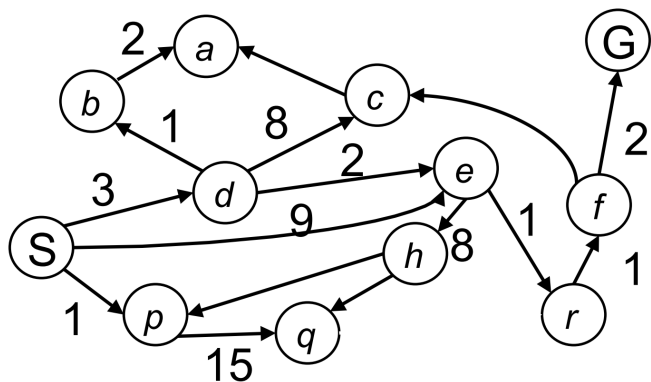
- 策略：每次取出容器中 **累计代价 $g(n)$** 最小的节点
 - $g(n)$: 从初始点到点 n 的累计代价
 - 更新节点 “ n ” 的所有未拓展邻居 “ m ” 的累计成本 $g(m)$
 - 已扩展节点的累计代价应为到起始点的最短路径代价



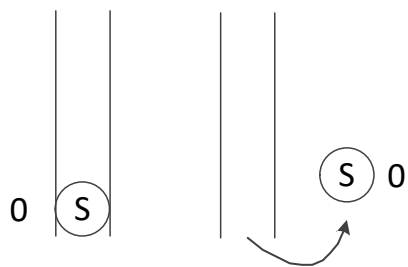
Dijkstra 算法

- 维护一个存储待扩展节点的**优先队列**
- 根据初始状态 X_s 初始队列
- 赋值 $g(X_s)=0$, 且对于图中其他节点 $g(n)=\infty$
- 循环
 - 如果队列为空, 返回FALSE; 退出循环
 - 从优先队列中移出最小 $g(n)$ 的节点“n”
 - 将节点“n”记作已扩展的节点
 - 如果节点“n”是终点, 返回TRUE; 退出循环
 - 对于所有未扩展的节点“n”的邻居节点“m”
 - 如果 $g(m) = \infty$
 - $g(m) = g(n) + C_{nm}$
 - 将节点“m”压入队列
 - 如果 $g(m) > g(n) + C_{nm}$
 - $g(m) = g(n) + C_{nm}$
 - end
- 结束循环

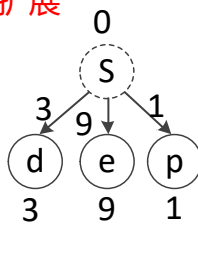
Dijkstra 算法



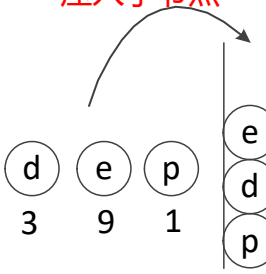
初始容器 取出g(n)值最小的节点



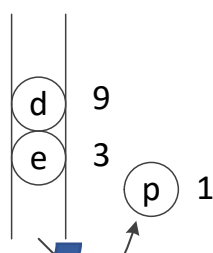
扩展



压入子节点



取出g(n)值最小的节点



循环



Dijkstra 算法

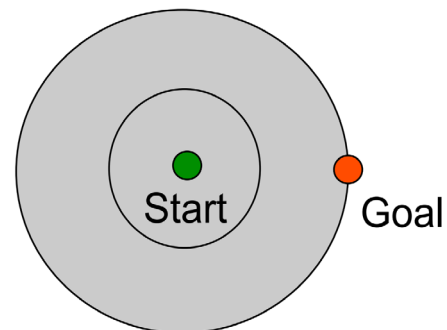
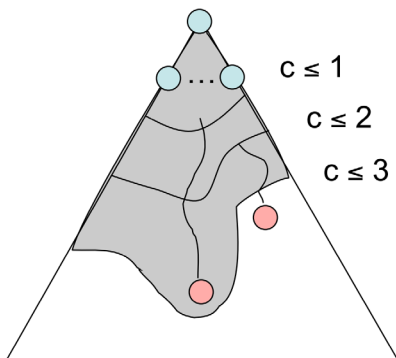
- 算法优缺点

- 优点:

- 具有完备性和最优性

- 缺点:

- 扩展不具有方向性，没有利用到目标点的信息





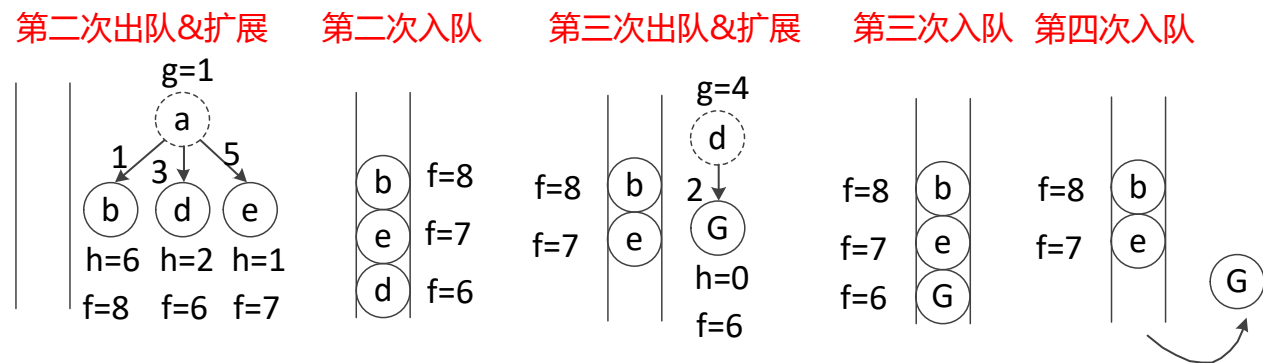
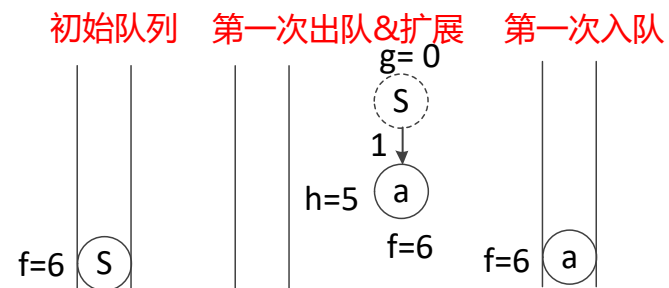
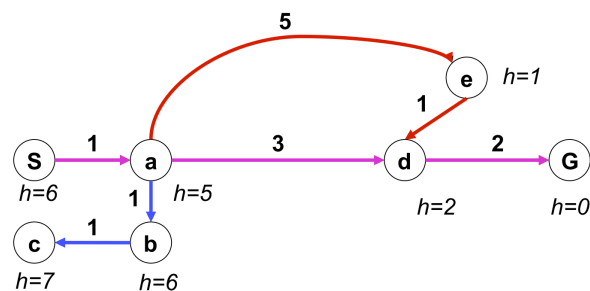
A*: Dijkstra算法+启发式函数

- 累计代价
 - $g(n)$: 从起始状态到节点“n”的最小估计代价
- 启发式函数
 - $h(n)$: 从节点到目标点的最小估计代价
- 从起始状态到目标状态，经过节点“n”的最小估计代价为
$$f(n) = g(n) + h(n)$$
- 策略：取出具有最小 $f(n)$ 的节点

A*算法

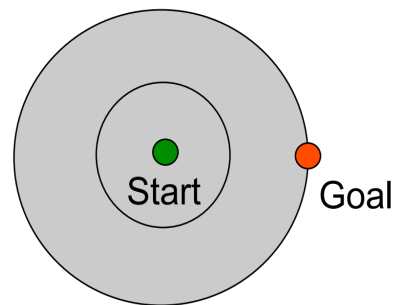
- 维护一个存储待扩展节点的**优先队列**
- 预先定义所有节点的启发式函数 $h(n)$
- 根据初始状态 X_S 初始队列
- 赋值 $g(X_S)=0$, 且对于图中其他节点 $g(n)=\infty$
- 循环
 - 如果队列为空, 返回FALSE; 退出循环 **和Dijkstra算法的唯一区别**
 - 从优先队列中**移出**最小 $f(n)=g(n)+h(n)$ 的节点“n”
 - 将节点“n”记作**已扩展**的节点
 - 如果节点“n”是终点, 返回TRUE; 退出循环
 - 对于所有**未扩展**的节点“n”的邻居节点“m”
 - 如果 $g(m) = \infty$
 - $g(m) = g(n) + C_{nm}$
 - 将节点“m”压入队列
 - 如果 $g(m) > g(n) + C_{nm}$
 - $g(m) = g(n) + C_{nm}$
 - end
- 结束循环

A*算法

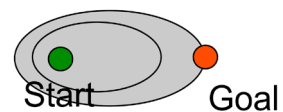


Dijkstra vs. A*

- Dijkstra算法朝各个方向探索

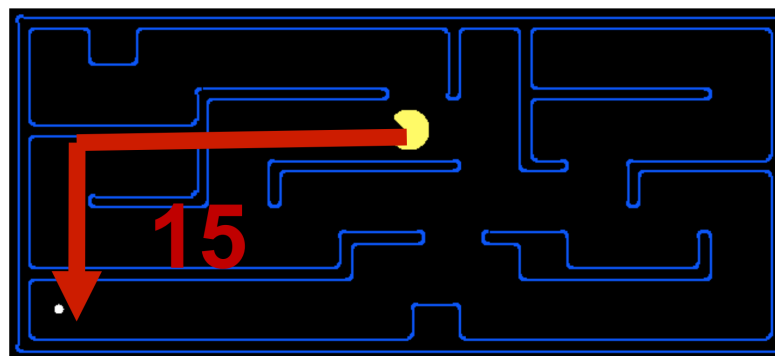
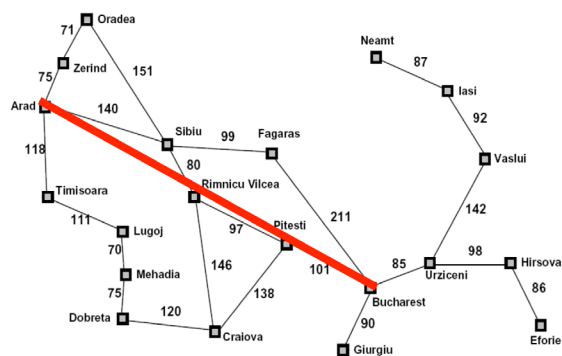


- A*算法主要朝着目标点方向探索



可采用的启发式函数

- 启发式函数 h 是可采用的 (**admissible**) 如果:
 - 所有的节点 $h(n) \leq h^*(n)$, $h^*(n)$ 是从节点 n 到终点的真实最小距离
- 如果启发式函数可采用, 那么A*搜索是最优的
- 实际中想出可采用的启发式函数是使用A*中的主要环节
- 举例:



次优解

使用过分估计的启发式函数会怎样？

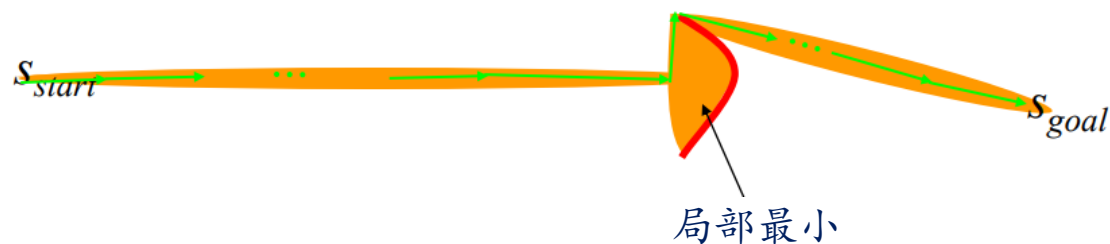


- 次优路径
- 更快



加权A*:
(Weighted A*)

根据 $f = g + \epsilon h, \epsilon > 1$ 拓展搜索，
更倾向于靠近终点的状态



- 加权A* 搜索:

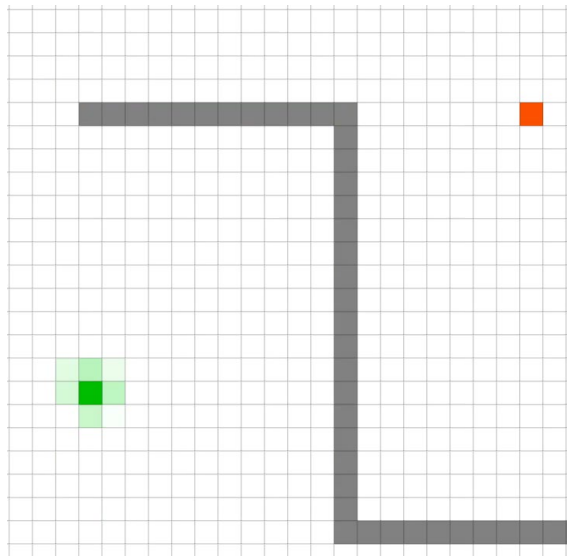
- 最优性 vs. 速度
- ϵ -suboptimal
- 比A*快几个数量级

加权A* $\rightarrow$ Anytime A* $\rightarrow$ ARA* $\rightarrow$ D*

超过本课程内容

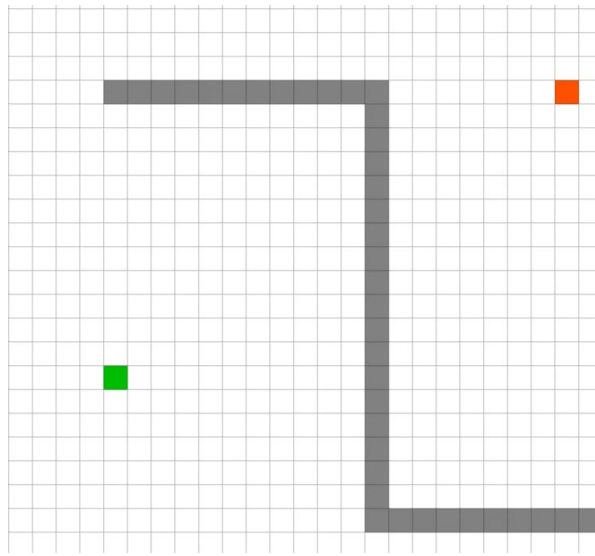
贪婪优先搜索 vs. 加权 A^* vs. A^*

$$f = a \cdot g + b \cdot h$$



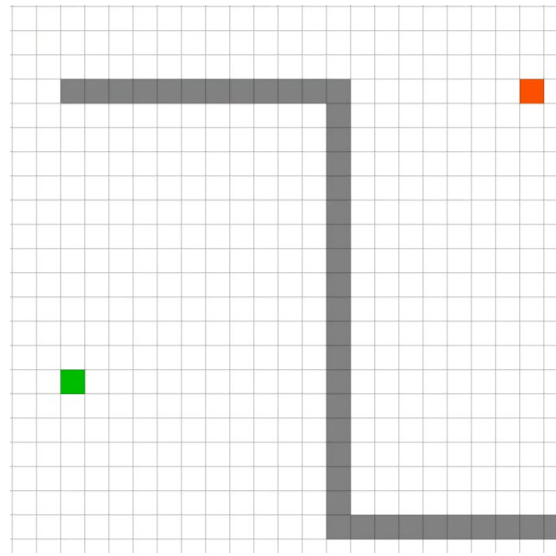
最贪婪的情况

$$a = 0, b = 1$$



可调整的贪婪

$$a = 1, b = \varepsilon > 1$$



最优

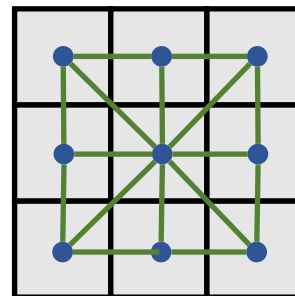
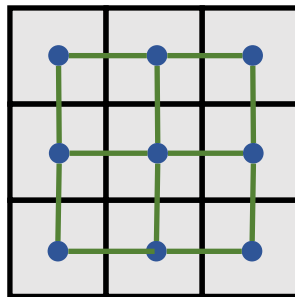
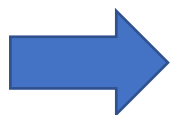
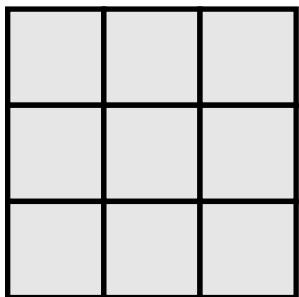
$$a = 1, b = 1$$

Dijkstra: $a = 1, b = 0$

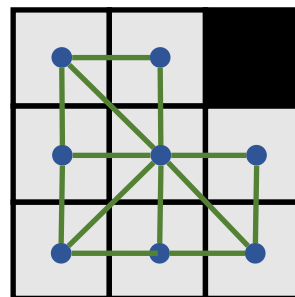
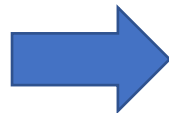
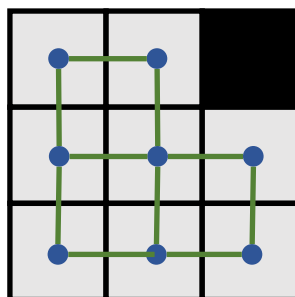
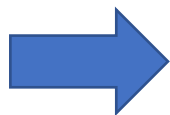
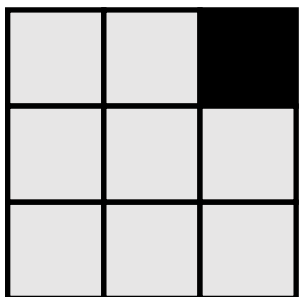
举例：基于栅格的路径搜索

怎么把栅格表示为图？

每个单元是一个节点，边连接相邻的单元



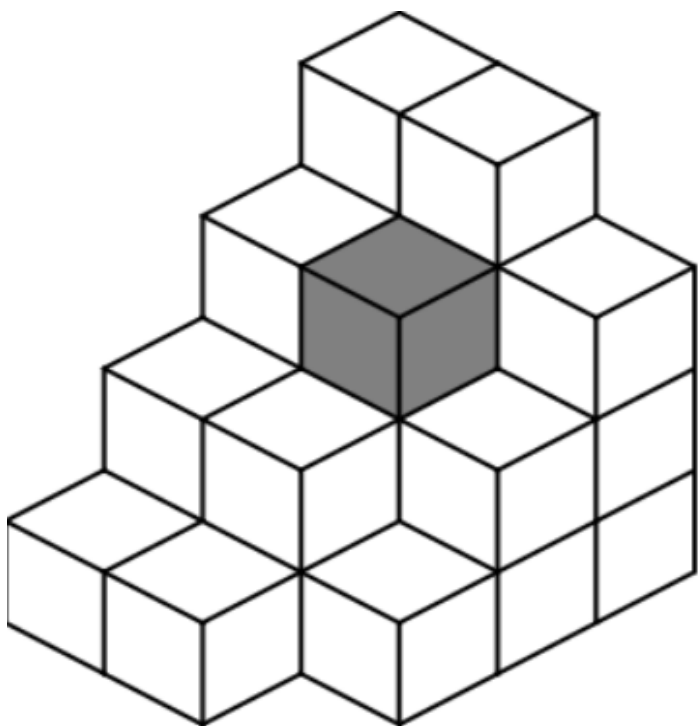
常用选择！



4 方向连接

8 方向连接

基于栅格的路径搜索：补充



- 构建稠密图
 - 连接栅格地图中被占据的栅格
 - 根据栅格索引寻找相邻节点
 - 使用A*搜索
-
- **Priority queue in C++**
 - `std::priority_queue`
 - `std::make_heap`
 - `std::multimap`

最佳启发式函数

• 回忆：以下距离表示是否可采用？

- 欧式距离 (L2范数)
- 曼哈顿距离 (L1范数)
- L^∞ 范数
- 0距离

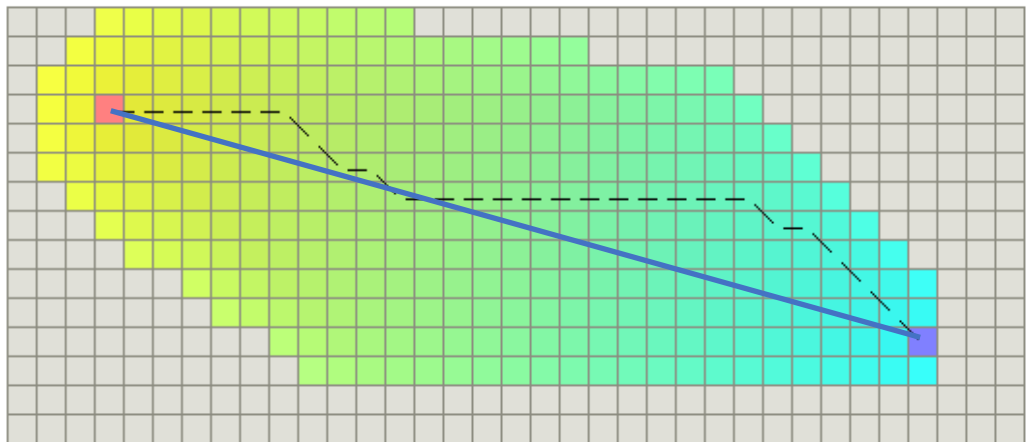


它们可用，但都不是最佳选择，why？

因为他们都不紧凑 (Tight)

紧凑意味着衡量的距离与真实最短距离的相近程度

欧式距离启发式函数



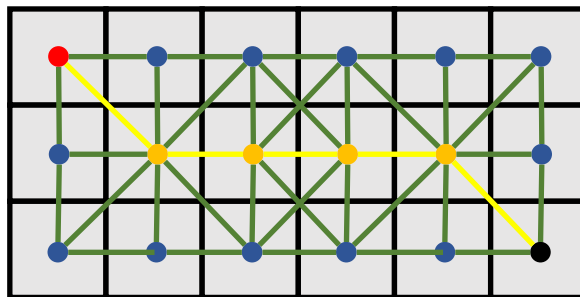
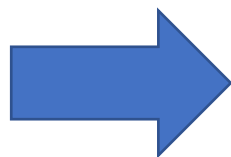
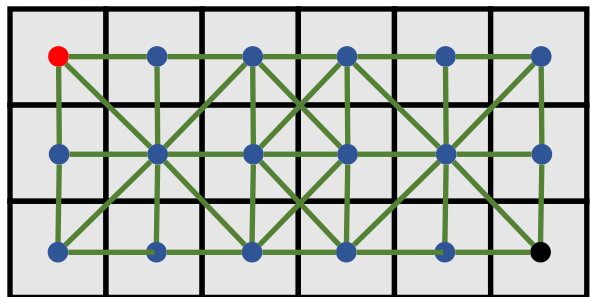
为什么拓展了这么多节点？

因为欧式距离与理论上的最短距离相差很大

最佳启发式函数

怎样获得真实的理论最优解？

栅格地图具备高度的结构性



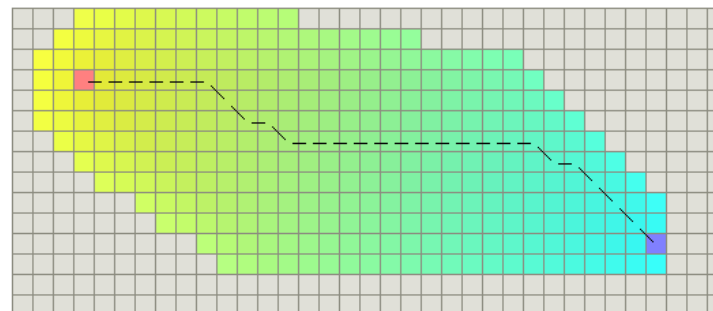
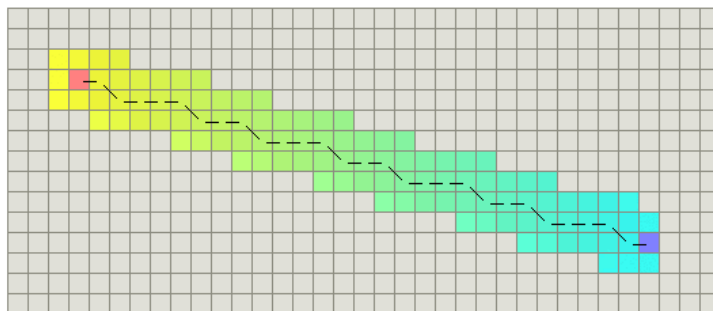
- 不需要搜索路径
- 具备闭式解！
 $dx = \text{abs}(\text{node.x} - \text{goal.x})$
 $dy = \text{abs}(\text{node.y} - \text{goal.y})$
 $h = (dx + dy) + (\sqrt{2} - 2) * \min(dx, dy)$



3维场景也类似

比较

对角启发式函数



Tie Breaker

Tie breaker核心观点:

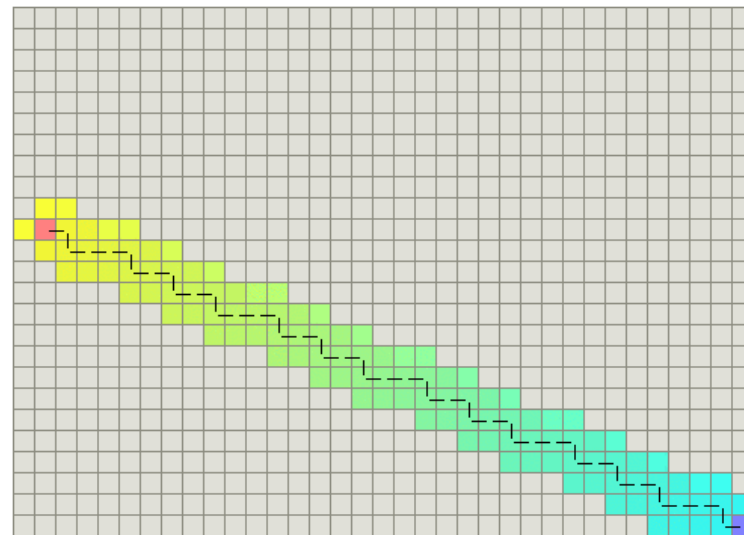
在相同cost的路径中找到某种倾向

- 当 f 相同, 比较 h .
- 在启发式函数或边的cost中增加确定的随机数 (关于坐标的哈希)
- 倾向于从起点到终点的直线路径

$$\begin{aligned}dx1 &= \text{abs}(\text{node}.x - \text{goal}.x) \\dy1 &= \text{abs}(\text{node}.y - \text{goal}.y) \\dx2 &= \text{abs}(\text{start}.x - \text{goal}.x) \\dy2 &= \text{abs}(\text{start}.y - \text{goal}.y) \\cross &= \text{abs}(dx1 \times dy2 - dx2 \times dy1) \\h &= h + cross \times 0.001\end{aligned}$$

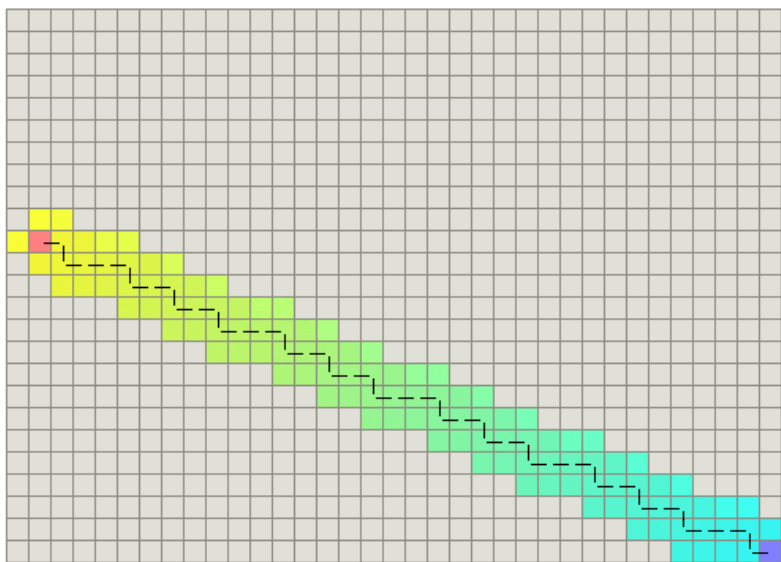
- ... 很多定制的方法

其他/更好的tie breaker?



Tie Breaker

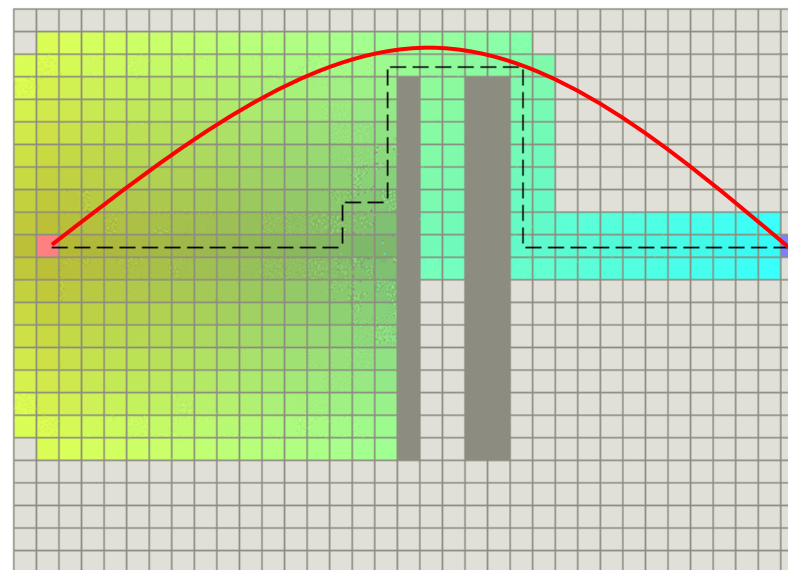
- 倾向于从起点到终点的直线路径



都是最优解，可是...

有些奇怪

看起来不错



最短路径，但是不利于轨迹生成（光滑性）

一种系统的方法：跳点搜索 Jump Point Search (JPS)

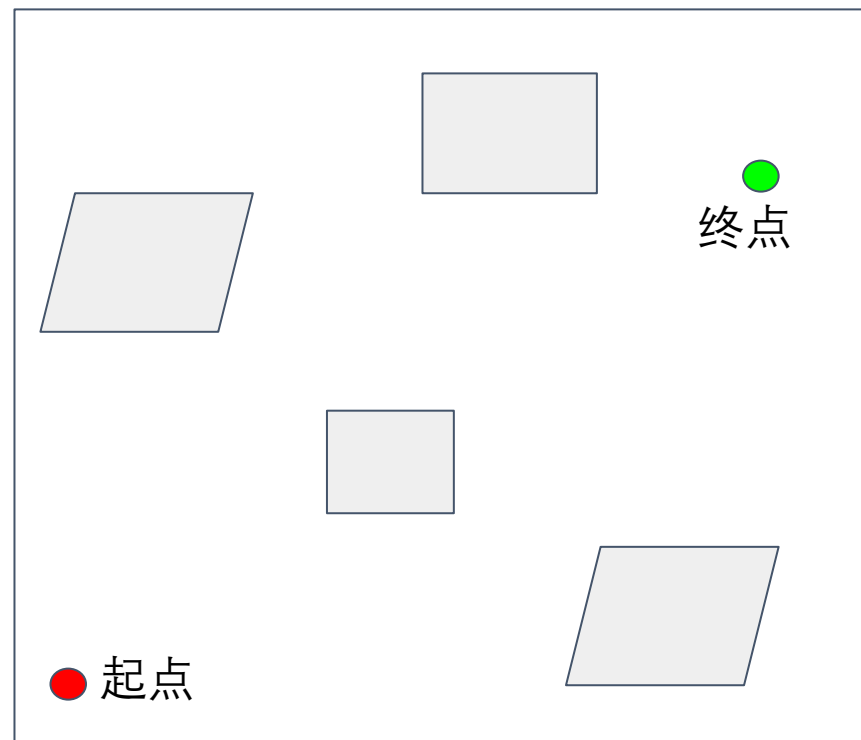
基本特点

- 高效地探索环境的拓扑结构，即可行区域的连接情况
- 不显示地构建构型空间及其边界
- 一般具有概率完备性
- 一般具有次优性或渐进最优性
- 可分为单查询（single-query）和多查询（multi-query）

算法介绍-PRM

PRM是什么？

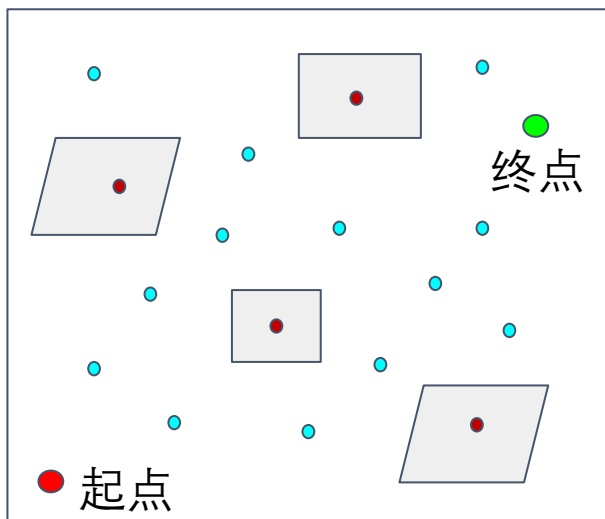
- 全称为Probabilistic Road Map
- 一种多查询（multi-query）算法
- 将规划分为两个阶段：
 - 构建阶段
 - 搜索阶段
- 用相对少量的路标点 and 局部路径来构建一个连接图以得到可行区域的连接情况



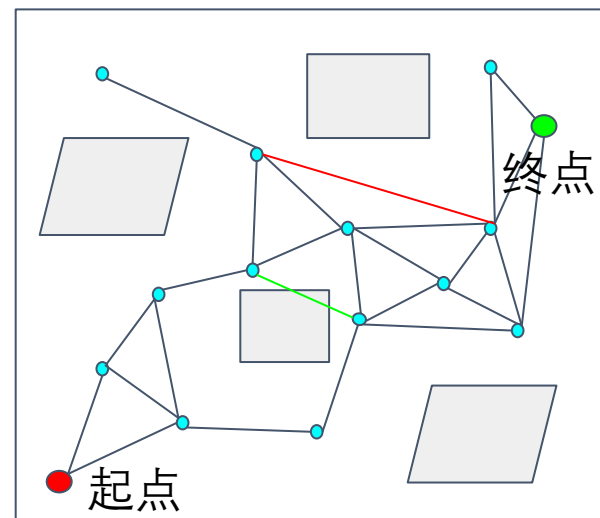
算法介绍-PRM

构建阶段：构建一个表征环境连通情况的路标图

- 在工作空间采样N个点
- 删除和环境碰撞的点



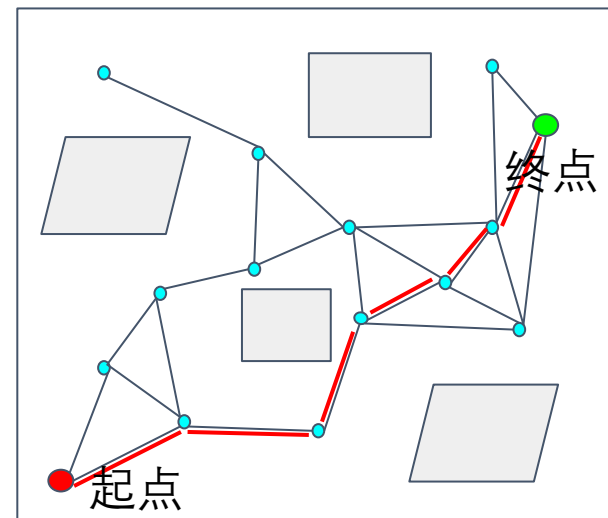
- 连接近邻的节点
- 删除和环境碰撞的路径段



算法介绍-PRM

搜索阶段

- 在构建出的路标连接图中搜索一条起点到终点的路径（使用Dijkstra或者A*算法）
- 路标图可近似看作栅格图
- 可利用路标图进行多次搜索（multi-query）



算法介绍-PRM

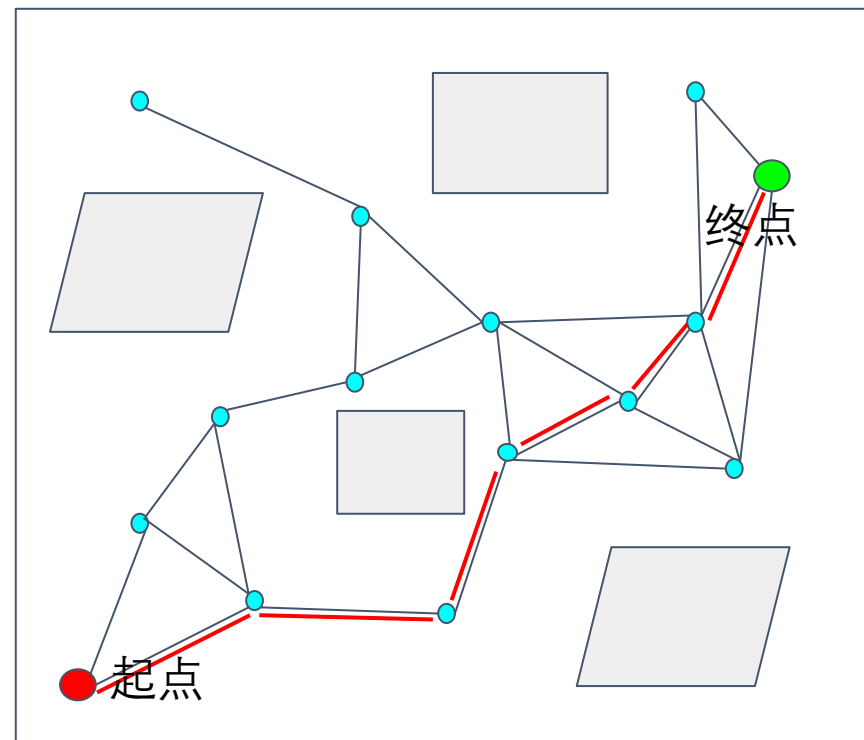
算法优劣

优势

- 概率完备性

劣势

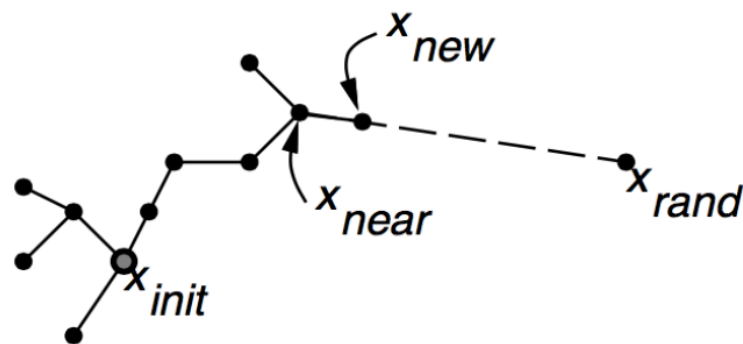
- 构建连接图但没有专注于产生路径
- 低效



算法介绍-RRT

RRT是什么？

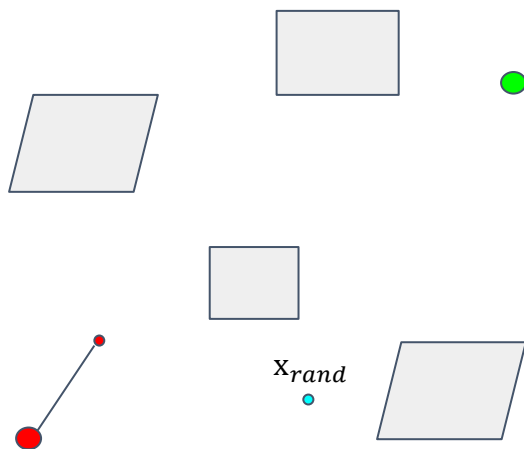
- 全称为Rapidly-exploring Random Trees
- 一种单查询（single-query）算法
- 通过在工作空间采样节点来构建一棵从起点到终点的树，随着采样增加，树从起点向终点生长



算法介绍-RRT

算法流程

- 在可行空间随机采样



Algorithm 1: RRT Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ to n **do**

$x_{rand} \leftarrow Sample(\mathcal{M});$

$x_{near} \leftarrow Near(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize);$

$E_i \leftarrow Edge(x_{new}, x_{near});$

if $CollisionFree(\mathcal{M}, E_i)$ **then**

$\mathcal{T}.addNode(x_{new});$

$\mathcal{T}.addEdge(E_i);$

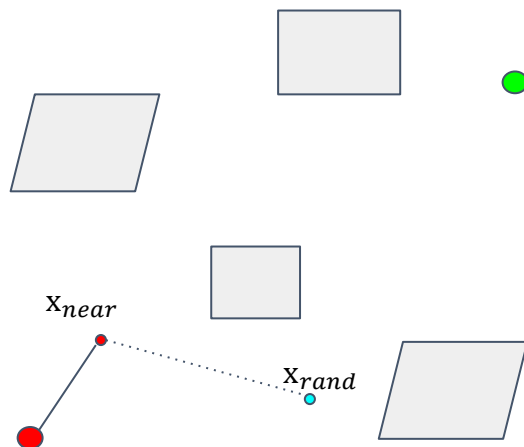
if $x_{new} = x_{goal}$ **then**

Success();

算法介绍-RRT

算法流程

- 找到当前树中离采样点最近的树节点



Algorithm 1: RRT Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

$E_i \leftarrow \text{Edge}(x_{new}, x_{near});$

if $\text{CollisionFree}(\mathcal{M}, E_i)$ **then**

$\mathcal{T}.addNode(x_{new});$

$\mathcal{T}.addEdge(E_i);$

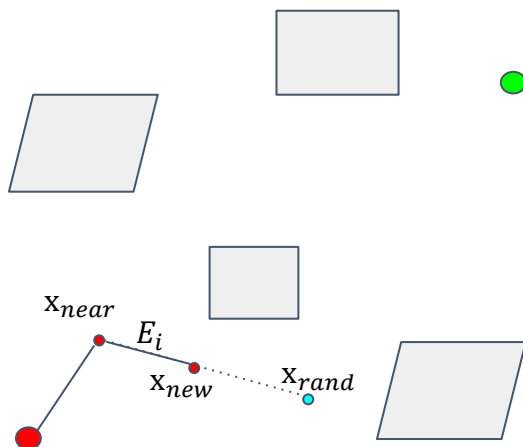
if $x_{new} = x_{goal}$ **then**

Success();

算法介绍-RRT

算法流程

- 从最近的树节点“生长”出新的节点和树枝（路径）



Algorithm 1: RRT Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

$E_i \leftarrow \text{Edge}(x_{new}, x_{near});$

if $\text{CollisionFree}(\mathcal{M}, E_i)$ **then**

$\mathcal{T}.addNode(x_{new});$

$\mathcal{T}.addEdge(E_i);$

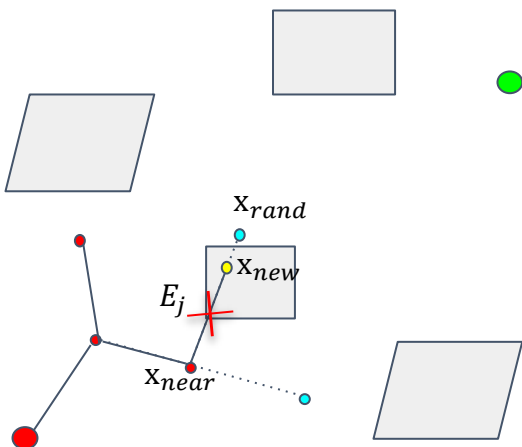
if $x_{new} = x_{goal}$ **then**

Success();

算法介绍-RRT

算法流程

- 如果此路径没有和环境发生碰撞, 则将此节点和路径加到树中



Algorithm 1: RRT Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

$E_i \leftarrow \text{Edge}(x_{new}, x_{near});$

if $\text{CollisionFree}(\mathcal{M}, E_i)$ **then**

$\mathcal{T}.addNode(x_{new});$

$\mathcal{T}.addEdge(E_i);$

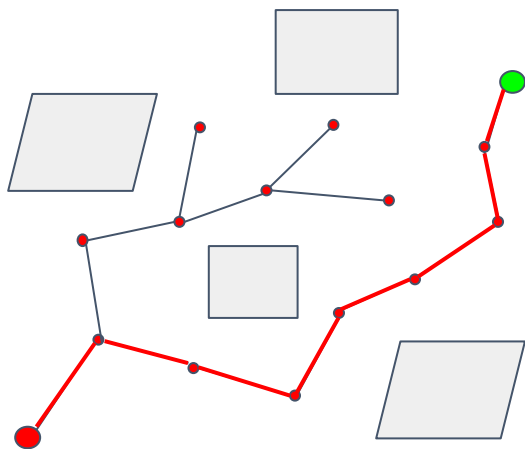
if $x_{new} = x_{goal}$ **then**

Success();

算法介绍-RRT

算法流程

- 重复n次采样，直到树生长到终点区域



Algorithm 1: RRT Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow Sample(\mathcal{M});$

$x_{near} \leftarrow Near(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize);$

$E_i \leftarrow Edge(x_{new}, x_{near});$

if $CollisionFree(\mathcal{M}, E_i)$ **then**

$\mathcal{T}.addNode(x_{new});$

$\mathcal{T}.addEdge(E_i);$

if $x_{new} = x_{goal}$ **then**

Success();

算法介绍-RRT

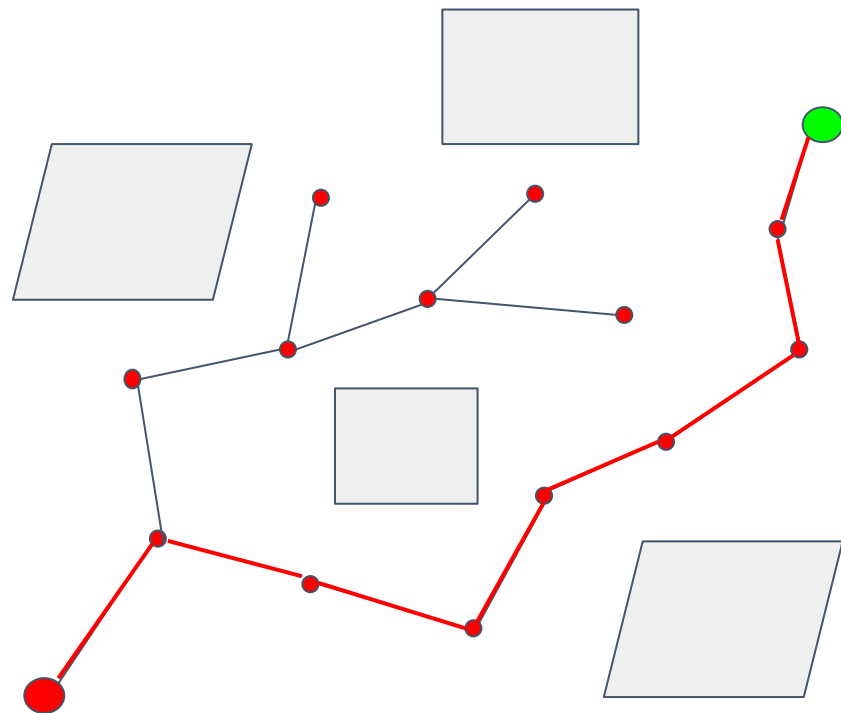
算法优劣

优势

- 致力于找到从起点到终点的路径
- 相比PRM更具目标导向性

劣势

- 产生的路径非最优
- 不够高效





算法介绍-RRT *

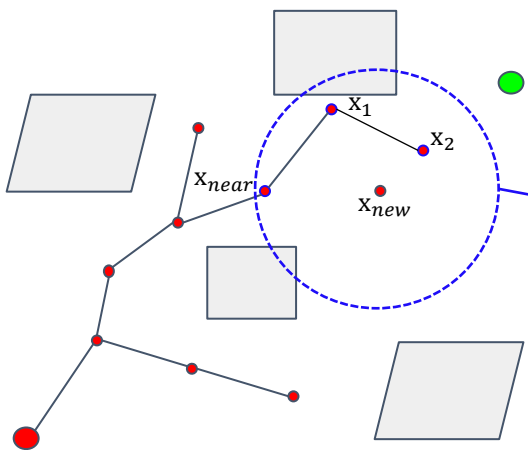
RRT*是什么？

- 对RRT的改进
- 具备概率完备性和渐进最优性
- 通过在工作空间采样节点来构建一棵从起点到终点的树，随着采样增加，树从起点向终点生长，并且改进已有路径

算法介绍- RRT *

改进之处

- 选择父节点时考虑多个邻近节点



Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

if $\text{CollisionFree}(x_{new})$ **then**

$X_{near} \leftarrow \text{NearC}(\mathcal{T}, x_{new});$

$x_{min} \leftarrow \text{ChooseParent}(X_{near}, x_{near}, x_{new});$

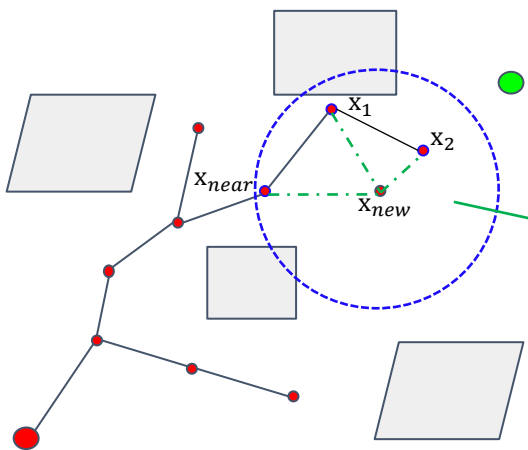
$\mathcal{T}.addNodeEdge(x_{min}, x_{new});$

$\mathcal{T}.rewire();$

算法介绍- RRT *

改进之处

- 考虑邻近节点的历史路径长度，而非只是局部路径长度



Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

if $\text{CollisionFree}(x_{new})$ **then**

$X_{near} \leftarrow \text{NearC}(\mathcal{T}, x_{new});$

$x_{min} \leftarrow \text{ChooseParent}(X_{near}, x_{near}, x_{new});$

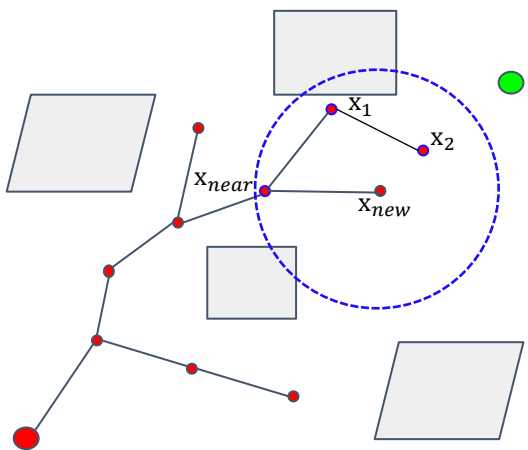
$\mathcal{T}.addNodeEdge(x_{min}, x_{new});$

$\mathcal{T}.rewire();$

算法介绍- RRT *

改进之处

- 考虑邻近节点的历史路径长度，而非只是局部路径长度



Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

if $\text{CollisionFree}(x_{new})$ **then**

$X_{near} \leftarrow \text{NearC}(\mathcal{T}, x_{new});$

$x_{min} \leftarrow \text{ChooseParent}(X_{near}, x_{near}, x_{new});$

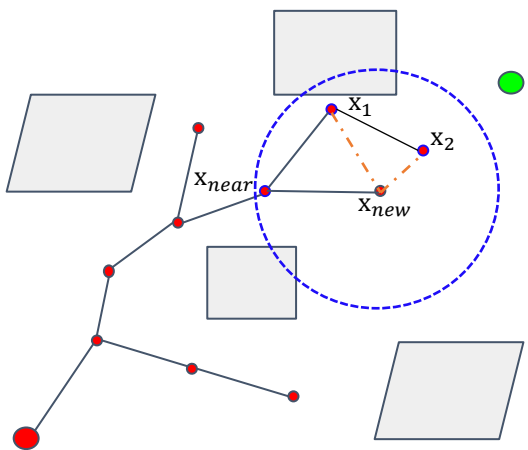
$\mathcal{T}.addNodeEdge(x_{min}, x_{new});$

$\mathcal{T}.rewire();$

算法介绍- RRT *

改进之处

- 重连接操作，改进局部最优解



Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}, x_{init}, x_{goal}$

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}.init();$

for $i = 1$ **to** n **do**

$x_{rand} \leftarrow \text{Sample}(\mathcal{M});$

$x_{near} \leftarrow \text{Near}(x_{rand}, \mathcal{T});$

$x_{new} \leftarrow \text{Steer}(x_{rand}, x_{near}, \text{StepSize});$

if $\text{CollisionFree}(x_{new})$ **then**

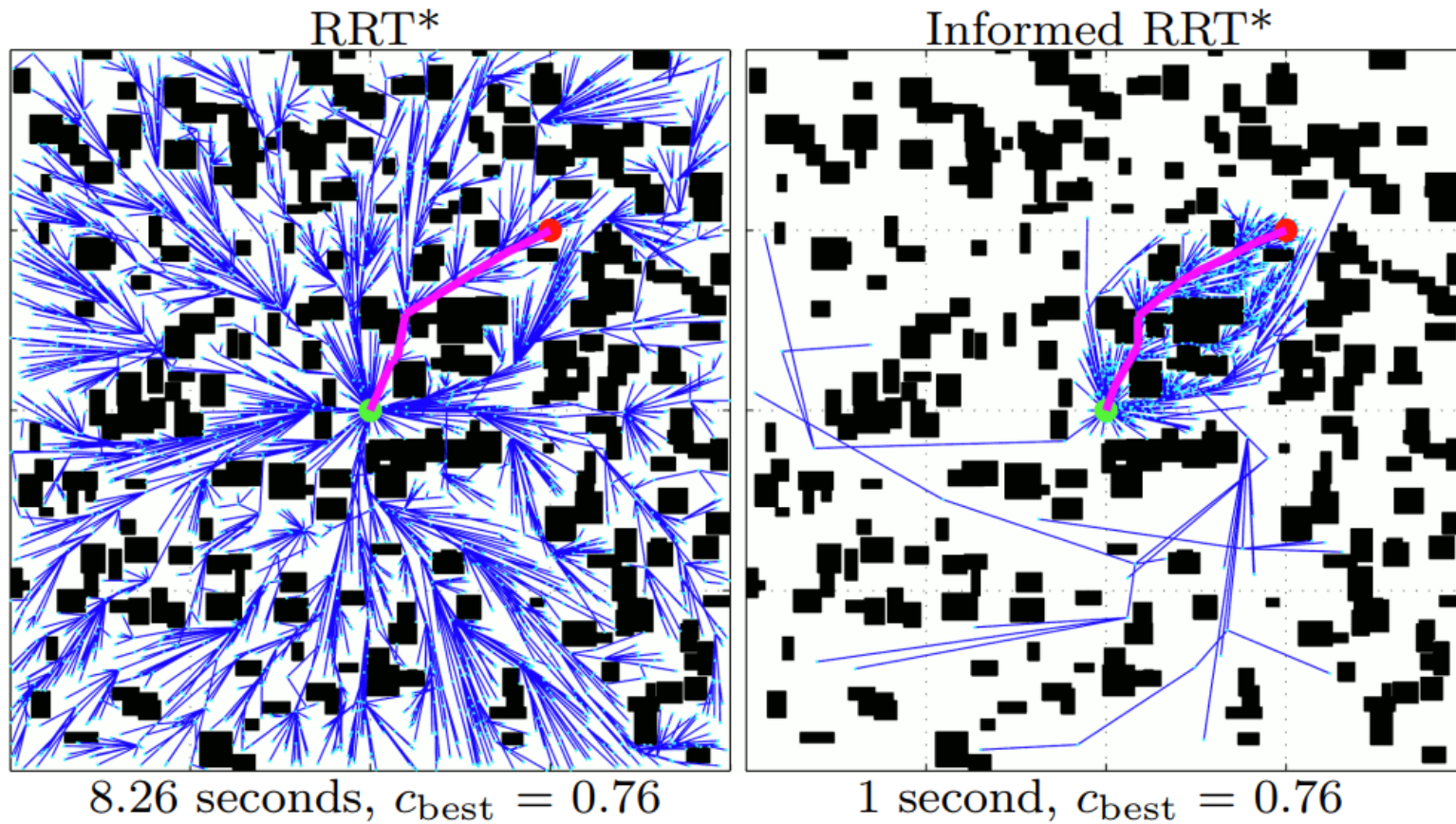
$X_{near} \leftarrow \text{NearC}(\mathcal{T}, x_{new});$

$x_{min} \leftarrow \text{ChooseParent}(X_{near}, x_{near}, x_{new});$

$\mathcal{T}.addNodeEdge(x_{min}, x_{new});$

$\mathcal{T}.rewire();$

启发式RRT*



Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic

路径规划->轨迹规划

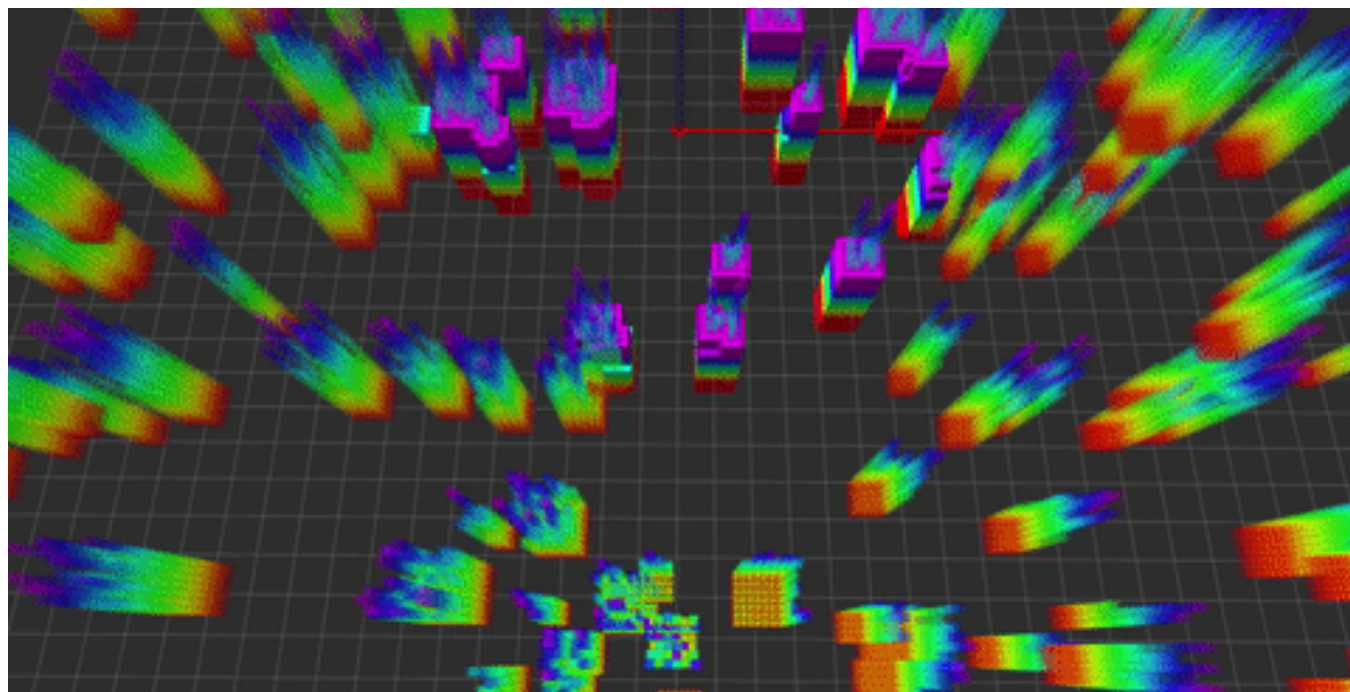
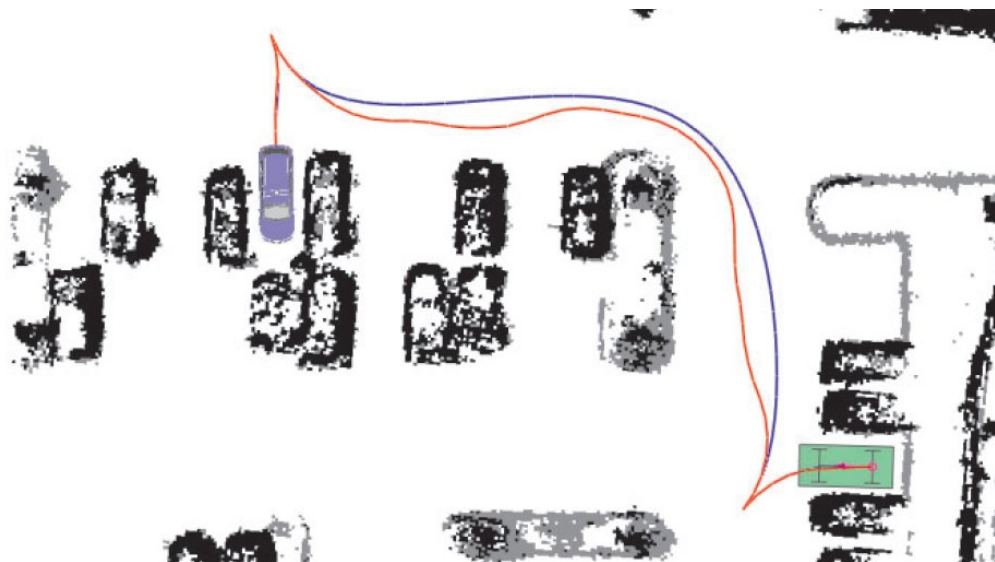
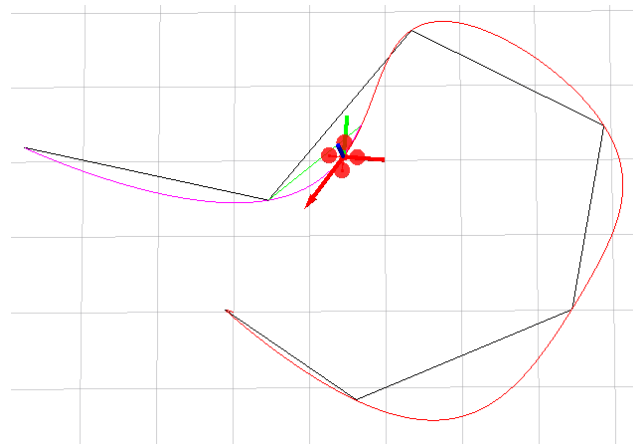


路径规划->轨迹规划



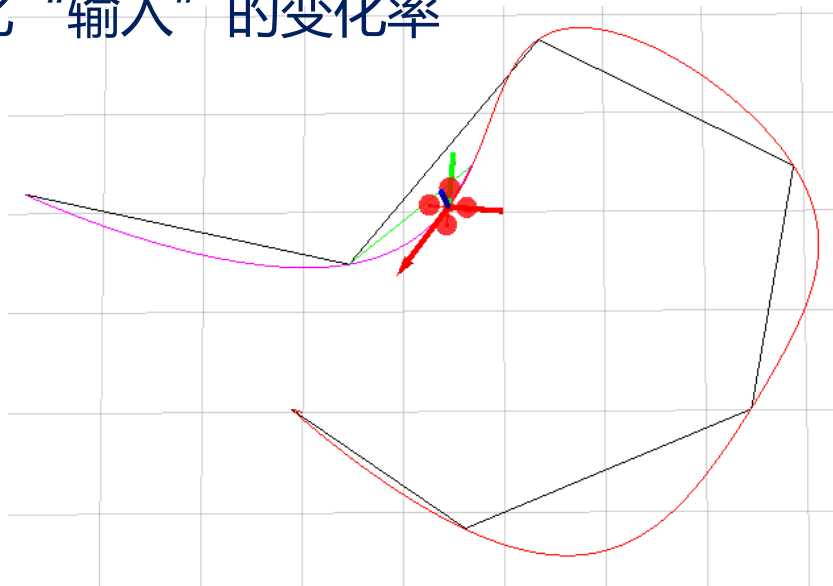
轨迹优化的必要性

- 机器人的速度和动力学高阶量无法突变；
- 节约机器人运动过程的能量消耗；
- 保证轨迹执行时间的合理性；
- 确保机器人移动过程中的安全；



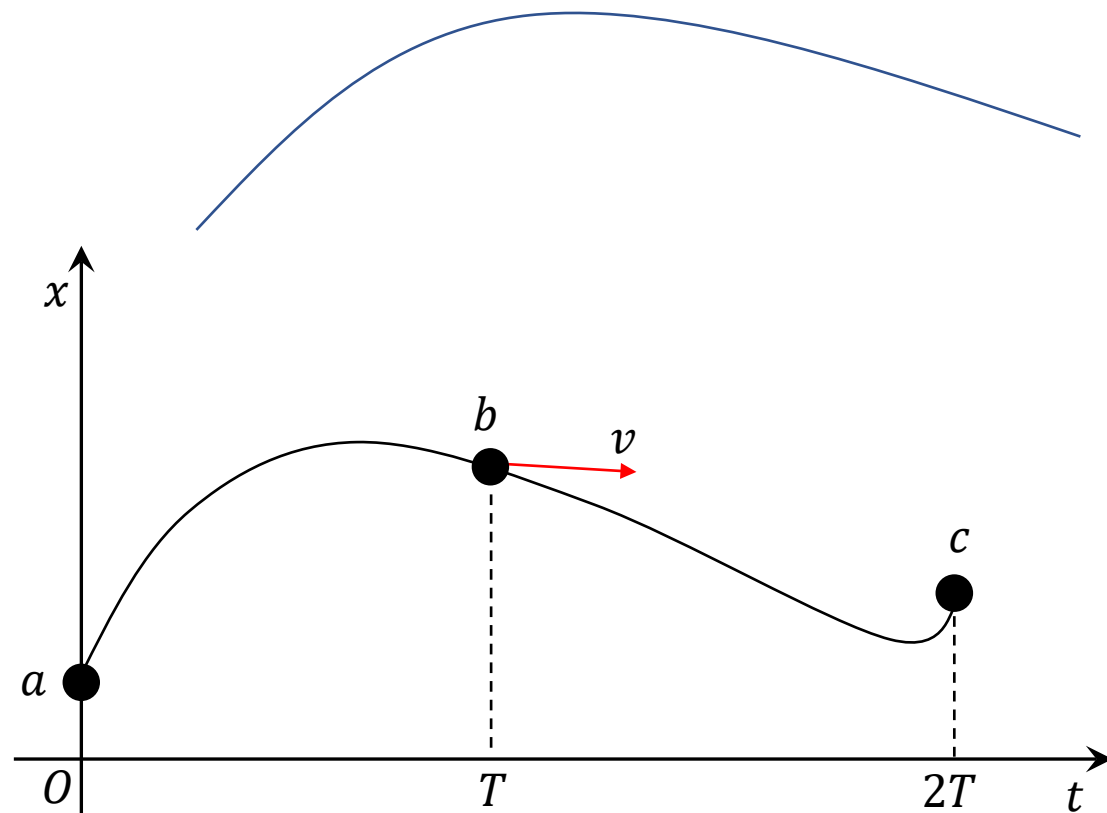
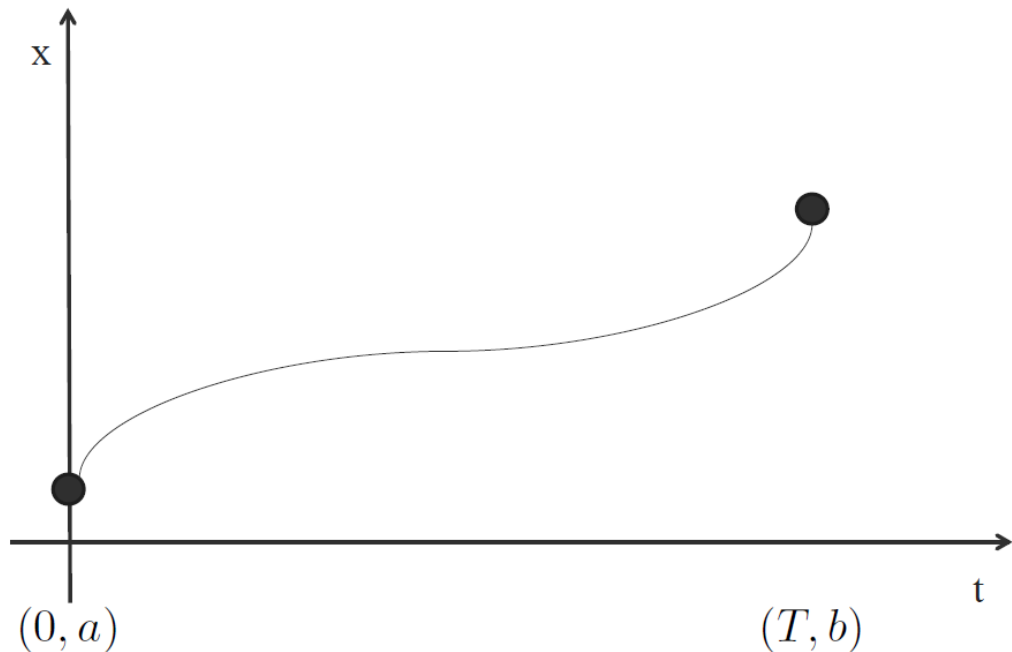
光滑轨迹生成

- 边界条件：起始位置（朝向）
- 中间条件：航点位置（朝向）
 - 航点由路径规划得到 (A^* , RRT*等)
 - 之前3节课的内容
- 光滑性的标准
 - 通常通过最小化“输入”的变化率



光滑一维轨迹

- 设计一条轨迹 $x(t)$ 使得:
 - $x(0) = a$
 - $x(T) = b$ } 边界条件
- 轨迹光滑性通过轨迹参数化形式保证



光滑一维轨迹

- 5th 多项式轨迹:

- $x(t) = p_5 t^5 + p_4 t^4 + p_3 t^3 + p_2 t^2 + p_1 t + p_0$



轨迹参数化

- 边界条件

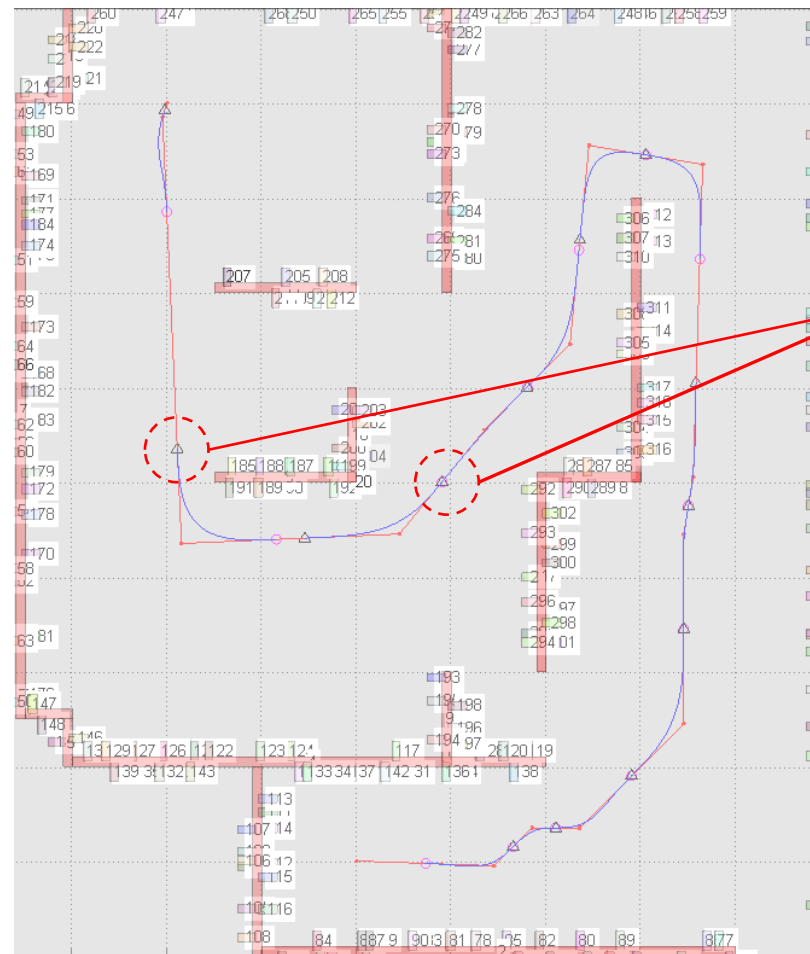
	位置	速度	加速度
t = 0	a	0	0
t = T	b	0	0

- 求解:

$$\begin{bmatrix} a \\ b \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ T^5 & T^4 & T^3 & T^2 & T & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 5T^4 & 4T^3 & 3T^2 & 2T & 1 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 20T^3 & 12T^2 & 6T & 2 & 0 & 0 \end{bmatrix} \begin{bmatrix} p_5 \\ p_4 \\ p_3 \\ p_2 \\ p_1 \\ p_0 \end{bmatrix}$$

光滑多段一维轨迹

- 光滑直线段的转角处
- 倾向于以 v 作匀速运动
- 倾向于零加速度
- 短的直线段需要特殊处理



中间条件

光滑多段一维轨迹

- 在每个维度上仍使用相同的参数化形式:

- $x(t) = p_5 t^5 + p_4 t^4 + p_3 t^3 + p_2 t^2 + p_1 t + p_0$

- 边界条件

	位置	速度	加速度
$t = 0$	a	v_0	0
$t = T$	b	v_T	0

- 求解:

$$\begin{bmatrix} a \\ b \\ v_0 \\ v_T \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ T^5 & T^4 & T^3 & T^2 & T & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 5T^4 & 4T^3 & 3T^2 & 2T & 1 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 20T^3 & 12T^2 & 6T & 2 & 0 & 0 \end{bmatrix} \begin{bmatrix} p_5 \\ p_4 \\ p_3 \\ p_2 \\ p_1 \\ p_0 \end{bmatrix}$$

微分平坦

微分平坦

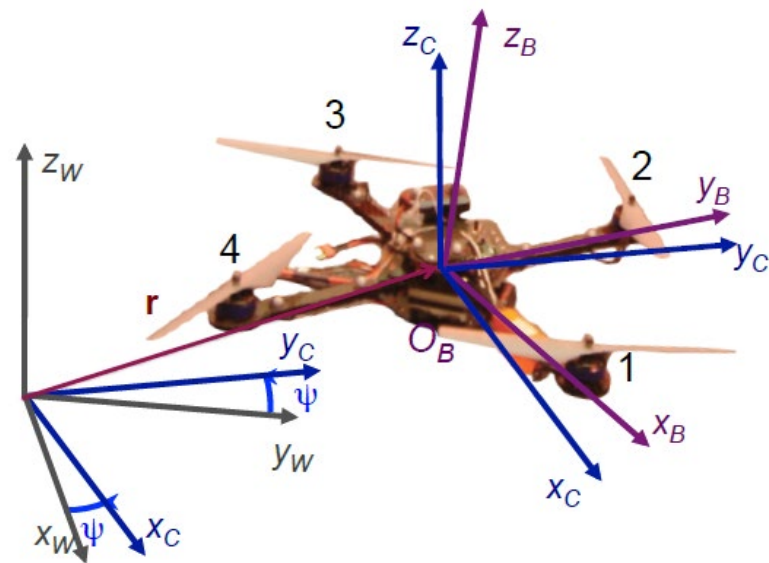
- 四旋翼的状态和输入可以写作四个平坦 (flat) 的输出变量和它们导数的代数方程
 - 使得轨迹能够自动生成
- 任何平坦的输出变量空间中的光滑轨迹 (其导数在合理范围内) 都能够被欠驱动的四旋翼系统跟随
- 一种可行的选择
 - $\sigma = [x, y, z, \psi]^T$
- 平坦输出空间中的轨迹
 - $\sigma(t) = [T_0, T_M] \rightarrow \mathbb{R}^3 \times SO(2)$

四旋翼动力特性

- 四旋翼状态

- 位置，朝向，线速度，角速度

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$



- 非线性动态方程

牛顿方程:
$$m\ddot{\mathbf{p}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ F_1 + F_2 + F_3 + F_4 \end{bmatrix} \quad \mathbf{u}_1$$

欧拉方程:
$$\mathbf{I} \cdot \begin{bmatrix} \dot{\omega}_x \\ \dot{\omega}_y \\ \dot{\omega}_z \end{bmatrix} + \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \times \mathbf{I} \cdot \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} l(F_2 - F_4) \\ l(F_3 - F_1) \\ M_1 - M_2 + M_3 - M_4 \end{bmatrix} \quad \begin{matrix} \mathbf{u}_2 \\ \mathbf{u}_3 \\ \mathbf{u}_4 \end{matrix}$$

微分平坦

- 四旋翼状态

- 位置, 朝向, 线速度, 角速度

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$

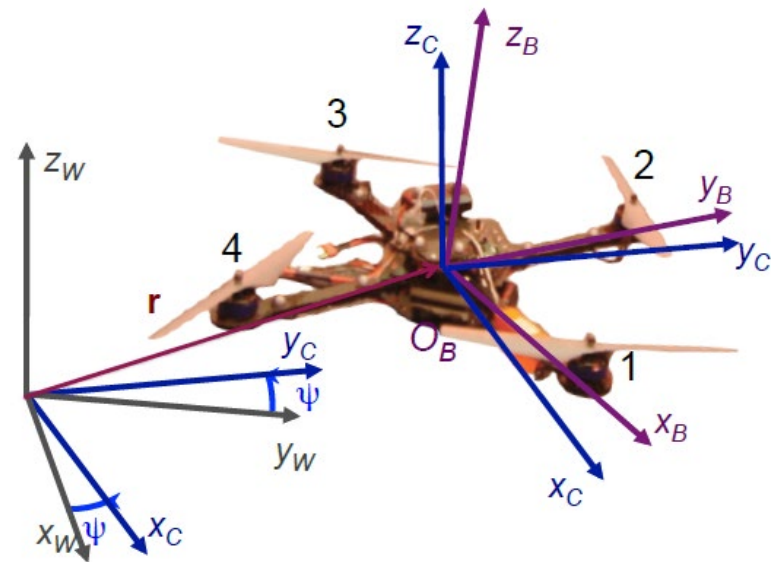
体坐标系中的体角加速度

- 运动方程

$$m\ddot{\mathbf{p}} = -mg\mathbf{z}_W + u_1\mathbf{z}_B.$$

$$\boldsymbol{\omega}_B = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}, \quad \dot{\boldsymbol{\omega}}_B = \mathbf{I}^{-1} \left[-\boldsymbol{\omega}_B \times \mathbf{I} \cdot \boldsymbol{\omega}_B + \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} \right]$$

- 位置, 速度和加速度是平坦输出的微分



微分平坦

• 朝向

- 四旋翼状态:

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$

- 从运动方程可得:

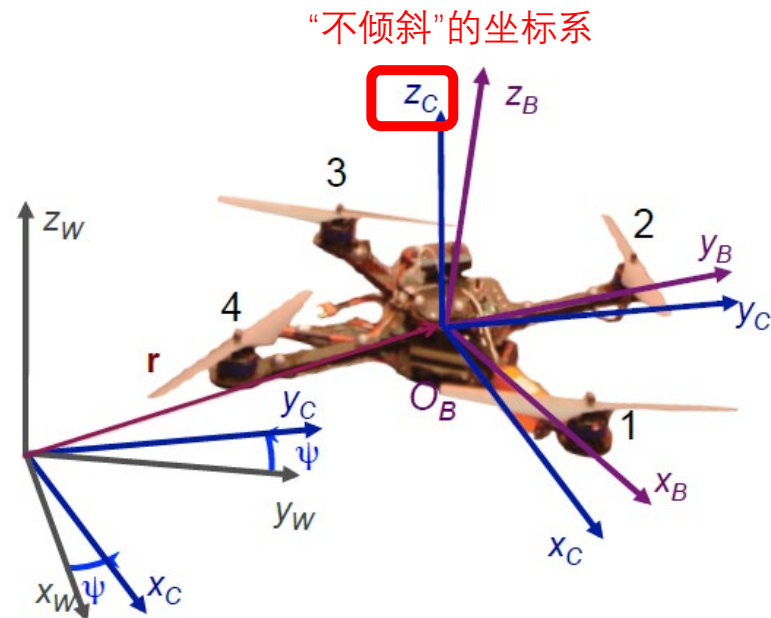
$$\mathbf{z}_B = \frac{\mathbf{t}}{\|\mathbf{t}\|}, \mathbf{t} = [\ddot{\sigma}_1, \ddot{\sigma}_2, \ddot{\sigma}_3 + g]^T$$

- 定义偏航角矢量 (Z-X-Y 欧拉角) :

$$\mathbf{x}_C = [\cos \sigma_4, \sin \sigma_4, 0]^T$$

- 姿态 (body坐标系朝向) 可以由平坦输出表示

$$\mathbf{y}_B = \frac{\mathbf{z}_B \times \mathbf{x}_C}{\|\mathbf{z}_B \times \mathbf{x}_C\|}, \quad \mathbf{x}_B = \mathbf{y}_B \times \mathbf{z}_B \quad \mathbf{R}_B = [\mathbf{x}_B \quad \mathbf{y}_B \quad \mathbf{z}_B]$$



$$\sigma = [x, y, z, \psi]^T$$

微分平坦

- 角速度

- 四旋翼状态:

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$

- 取运动方程的微分

$$m\ddot{\mathbf{p}} = -mg\mathbf{z}_W + u_1\mathbf{z}_B. \quad \longrightarrow \quad m\dot{\mathbf{a}} = \dot{u}_1\mathbf{z}_B + \boxed{\boldsymbol{\omega}_{BW}} \times u_1\mathbf{z}_B$$

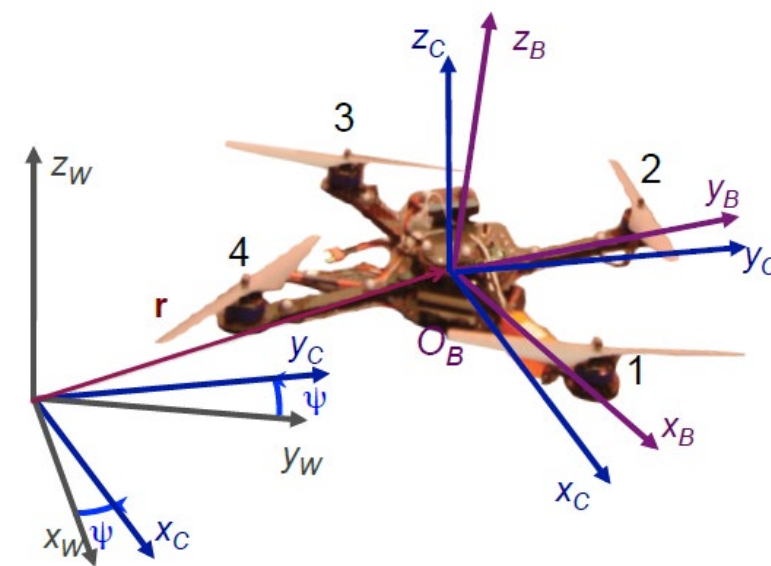
世界系中的体角速度

- 四旋翼仅有垂直的升力:

$$\dot{u}_1 = \mathbf{z}_B \cdot m\dot{\mathbf{a}}$$

- 可得到:

$$\mathbf{h}_\omega = \boldsymbol{\omega}_{BW} \times \mathbf{z}_B = \frac{m}{u_1} (\dot{\mathbf{a}} - (\mathbf{z}_B \cdot \dot{\mathbf{a}})\mathbf{z}_B).$$



微分平坦

- 角速度

- 四旋翼状态:

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$

- 已得到:

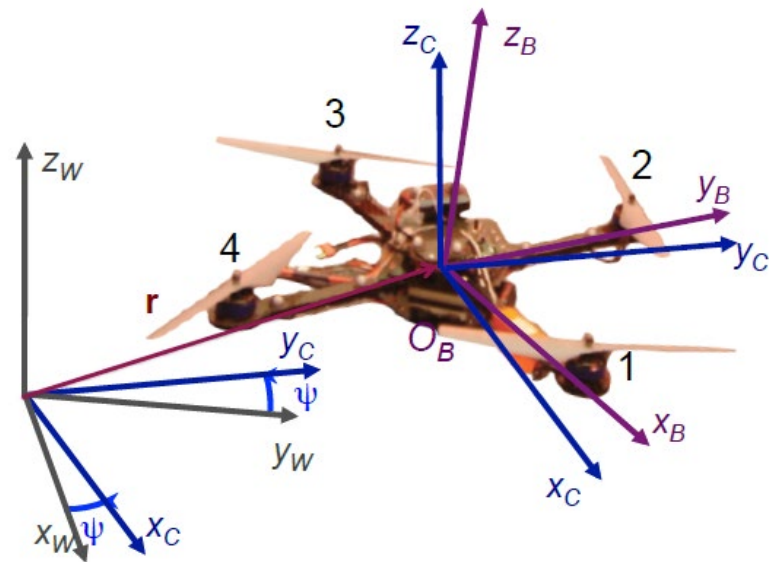
$$\mathbf{h}_\omega = \boldsymbol{\omega}_{BW} \times \mathbf{z}_B = \frac{m}{u_1} (\dot{\mathbf{a}} - (\mathbf{z}_B \cdot \dot{\mathbf{a}})\mathbf{z}_B).$$

- 又已知:

$$\boldsymbol{\omega}_{BW} = \omega_x \mathbf{x}_B + \omega_y \mathbf{y}_B + \omega_z \mathbf{z}_B, \quad \text{代入上式:}$$

- x_B 和 y_B 方向的角速度可以得到:

$$\omega_x = -\mathbf{h}_\omega \cdot \mathbf{y}_B, \quad \omega_y = \mathbf{h}_\omega \cdot \mathbf{x}_B$$



微分平坦

- 角速度

- 四旋翼状态:

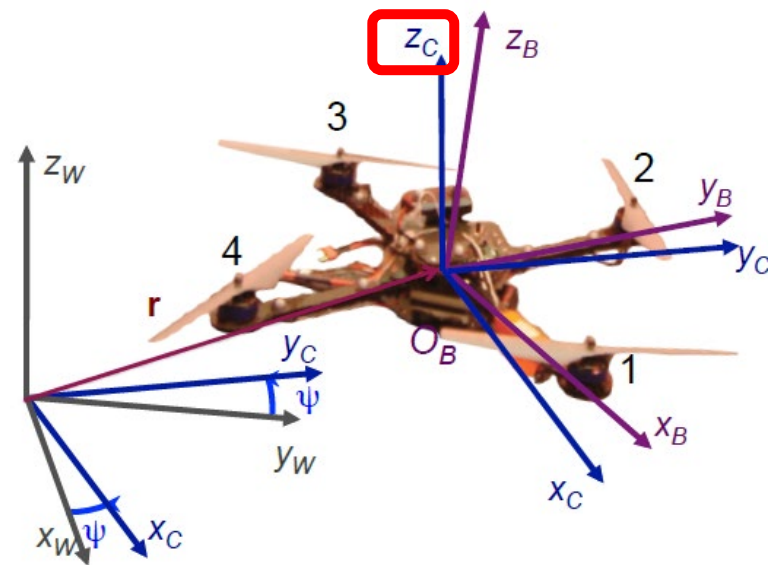
$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$

- 已得到:

$$\mathbf{h}_\omega = \boldsymbol{\omega}_{BW} \times \mathbf{z}_B = \frac{m}{u_1} (\dot{\mathbf{a}} - (\mathbf{z}_B \cdot \dot{\mathbf{a}}) \mathbf{z}_B).$$

- 因为 $\boldsymbol{\omega}_{BW} = \boldsymbol{\omega}_{BC} + \boldsymbol{\omega}_{CW}$, 且 $\boldsymbol{\omega}_{BC}$ 没有 $\mathbf{z}_B$ 的成分

$$\omega_z = \boldsymbol{\omega}_{BW} \cdot \mathbf{z}_B = \boldsymbol{\omega}_{CW} \cdot \mathbf{z}_B = \dot{\psi} \mathbf{z}_W \cdot \mathbf{z}_B.$$



微分平坦

- 总结

- 四旋翼状态:

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega_x, \omega_y, \omega_z]^T$$

- 平坦输出:

- $\sigma = [x, y, z, \psi]^T$

- 位置, 速度, 加速度:

- 平坦输出的微分

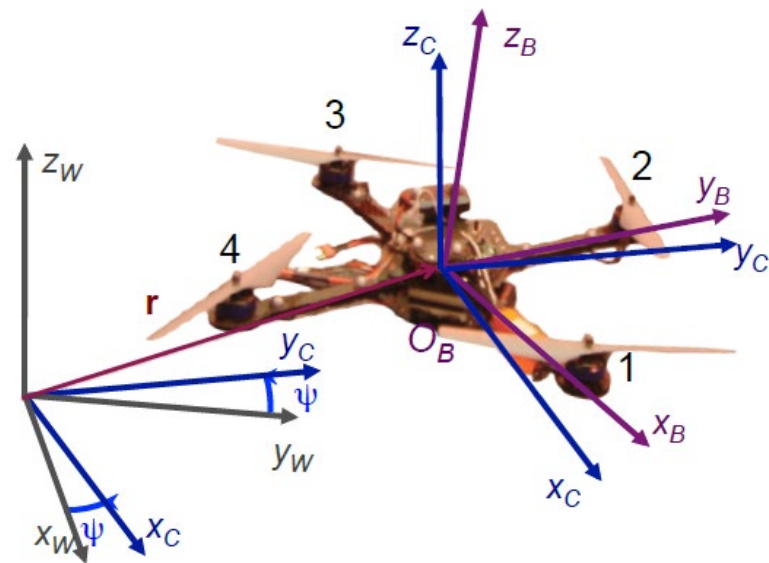
- 姿态:

$$\mathbf{x}_C = [\cos\sigma_4, \sin\sigma_4, 0]^T$$

$$\mathbf{R}_B = [\mathbf{x}_B \ \mathbf{y}_B \ \mathbf{z}_B]$$

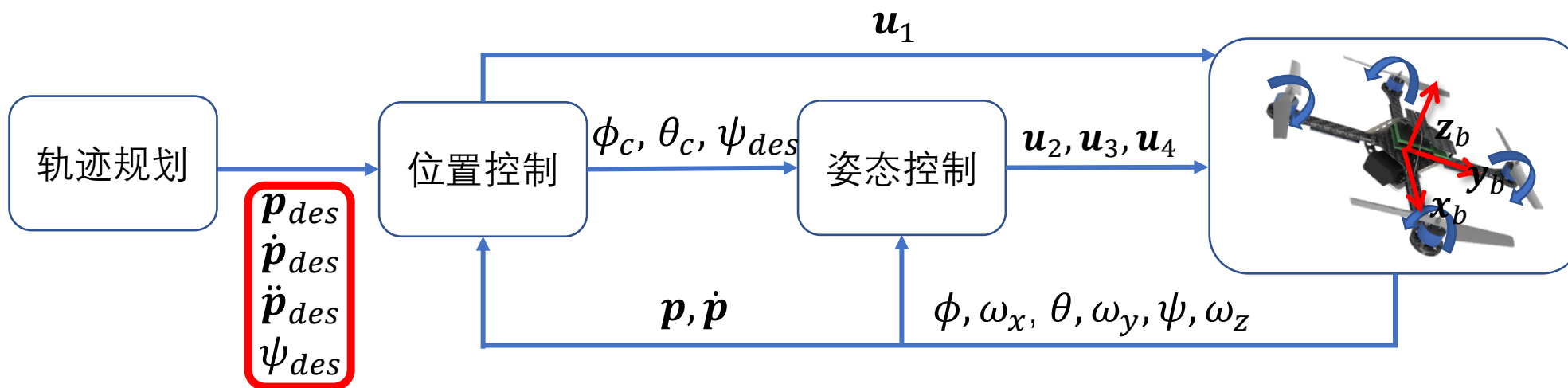
- 角速度:

$$\omega_x = -\mathbf{h}_\omega \cdot \mathbf{y}_B, \quad \omega_y = \mathbf{h}_\omega \cdot \mathbf{x}_B, \quad \omega_z = \dot{\psi} \mathbf{z}_W \cdot \mathbf{z}_B$$



→ 你只需要记住无人机的规划可以在 x, y, z, ψ 中进行。

规划-控制闭环



非线性动态方程

牛顿方程:
$$m\ddot{\mathbf{p}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ F_1 + F_2 + F_3 + F_4 \end{bmatrix}$$

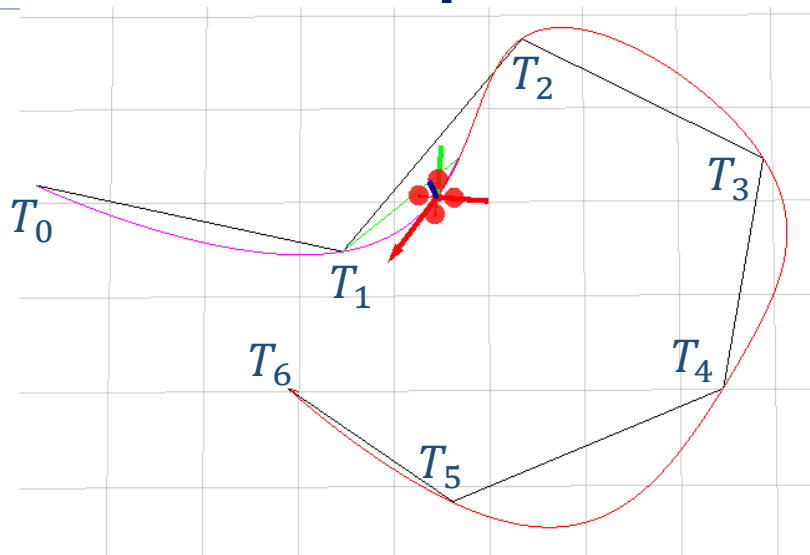
欧拉方程:
$$\mathbf{I} \cdot \begin{bmatrix} \dot{\omega}_x \\ \dot{\omega}_y \\ \dot{\omega}_z \end{bmatrix} + \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \times \mathbf{I} \cdot \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} l(F_2 - F_4) \\ l(F_3 - F_1) \\ M_1 - M_2 + M_3 - M_4 \end{bmatrix}$$

多项式轨迹

- 平坦输出：
 - $\sigma = [x, y, z, \psi]^T$
- 平坦输出空间中的轨迹：
 - $\sigma(t) = [T_0, T_M] \rightarrow \mathbb{R}^3 \times SO(2)$
- 多项式方程可用来指定平坦输出空间中的轨迹
 - 利用多项式的阶数判定平滑的标准
 - 简单且闭式地计算微分
 - 在三维中解耦轨迹生成

最小化Snap轨迹生成

最小化Snap的轨迹生成



问题定义：

- 轨迹的初始（或初末）状态给定；
- 轨迹在预定时刻需经过预设航点；
- 轨迹的4阶导数处处存在
- 最小化轨迹4阶导数平方幅值积分。

重要性质：

- 最优轨迹的参数化形式为分段多项式；
- 每段多项式的次数不超过 $2 \times 4 - 1 = 7$ ，即每段均可以被7次多项式表征。

$$f(t) = \begin{cases} f_1(t) \doteq \sum_{i=0}^N p_{1,i} t^i & T_0 \leq t \leq T_1 \\ f_2(t) \doteq \sum_{i=0}^N p_{2,i} t^i & T_1 \leq t \leq T_2 \\ \vdots & \vdots \\ f_M(t) \doteq \sum_{i=0}^N p_{M,i} t^i & T_{M-1} \leq t \leq T_M \end{cases}$$

最小化Snap的轨迹生成

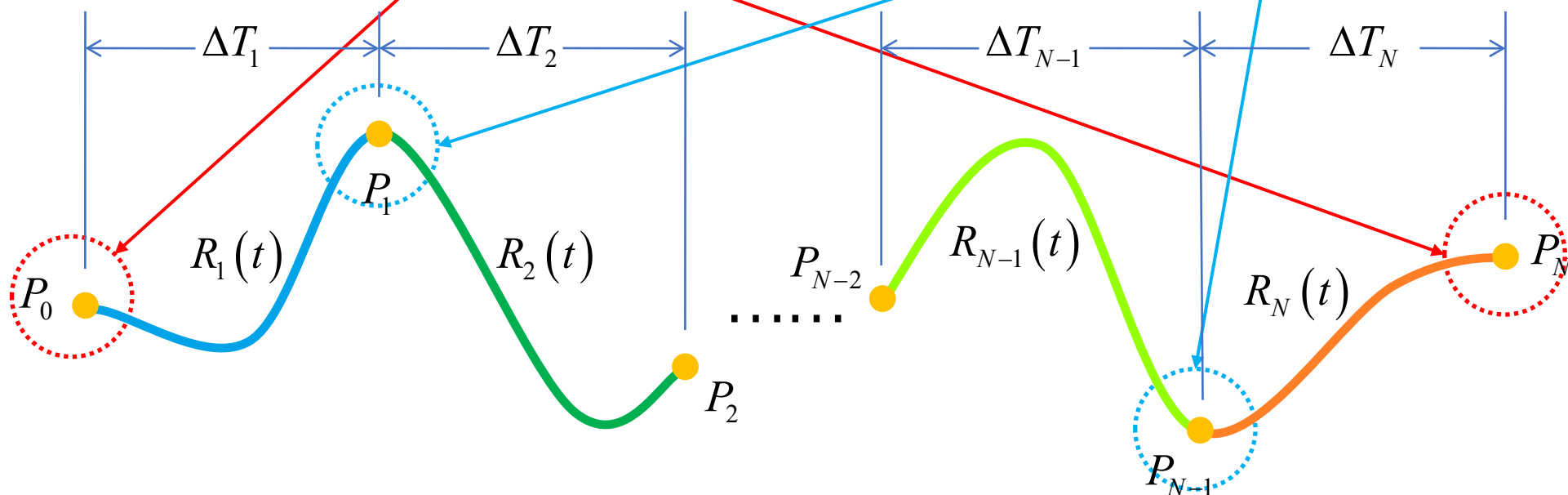
约束:

- 导数给定值约束:

$$\begin{cases} f_j^{(k)}(T_{j-1}) = x_{0,j}^{(k)} \\ f_j^{(k)}(T_j) = x_{T,j}^{(k)} \end{cases}$$

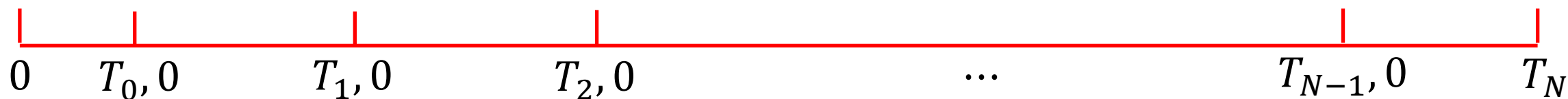
- 高阶连续性约束:

$$f_j^{(k)}(T_j) = f_{j+1}^{(k)}(T_j)$$

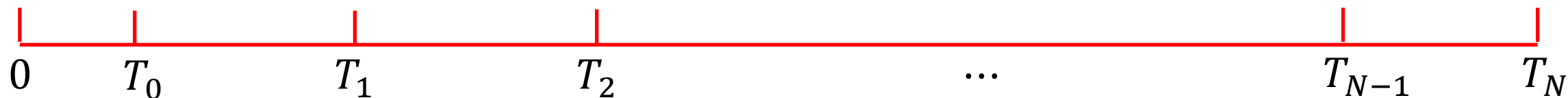


最小化Snap的轨迹生成

两种时间参数方案



方案1: 相对每一段初始的时间 优点: 数值稳定性高



方案2: 相对第一段初始的时间 优点: 数学表达清楚

最小化Snap的轨迹生成

目标函数的解析表达

半正定二次型

$$f(t) = \sum_i p_i t^i$$

$$J_j(T) = \mathbf{p}_j^T \mathbf{Q}_j \mathbf{p}_j$$

$$\Rightarrow f^{(4)}(t) = \sum_{i \geq 4} i(i-1)(i-2)(i-3)t^{i-4}p_i$$

$$\Rightarrow \left(f^{(4)}(t)\right)^2 = \sum_{i \geq 4, l \geq 4} i(i-1)(i-2)(i-3)l(l-1)(l-2)(l-3)t^{i+l-8}p_i p_l$$

$$\Rightarrow J(T) = \int_{T_{j-1}}^{T_j} \left(f^{(4)}(t)\right)^2 dt = \sum_{i \geq 4, l \geq 4} \frac{i(i-1)(i-2)(i-3)l(l-1)(l-2)(l-3)}{i+l-7} (T_j^{i+l-7} - T_{j-1}^{i+l-7}) p_i p_l$$

$$\Rightarrow J(T) = \int_{T_{j-1}}^{T_j} \left(f^{(4)}(t)\right)^2 dt = \begin{bmatrix} \vdots \\ p_i \\ \vdots \end{bmatrix}^T \begin{bmatrix} \vdots & & \\ \dots & \frac{i(i-1)(i-2)(i-3)l(l-1)(l-2)(l-3)}{i+l-7} T^{i+l-7} & \dots \\ \vdots & & \end{bmatrix} \begin{bmatrix} \vdots \\ p_l \\ \vdots \end{bmatrix}$$

最小化Snap的轨迹生成

指定导数值约束：0阶导给定即航点坐标

- $f_j^{(k)}(T_j) = x_j^{(k)}$

$$\Rightarrow \sum_{i \geq k} \frac{i!}{(i-k)!} T_j^{i-k} p_{j,i} = x_{T,j}^{(k)}$$

$$\Rightarrow \begin{bmatrix} \cdots & \frac{i!}{(i-k)!} T_j^{i-k} & \cdots \end{bmatrix} \begin{bmatrix} \vdots \\ p_{j,i} \\ \vdots \end{bmatrix} = x_{T,j}^{(k)}$$

$$\Rightarrow \begin{bmatrix} \cdots & \frac{i!}{(i-k)!} T_{j-1}^{i-k} & \cdots \\ \cdots & \frac{i!}{(i-k)!} T_j^{i-k} & \cdots \end{bmatrix} \begin{bmatrix} \vdots \\ p_{j,i} \\ \vdots \end{bmatrix} = \begin{bmatrix} x_{0,j}^{(k)} \\ x_{T,j}^{(k)} \end{bmatrix}$$

$$\Rightarrow \mathbf{A}_j \mathbf{p}_j = \mathbf{d}_j$$

$$x(t) = p_5 t^5 + p_4 t^4 + p_3 t^3 + p_2 t^2 + p_1 t + p_0$$

$$x(0) = \cdots, x(T) = \cdots$$

$$\dot{x}(0) = \cdots, \dot{x}(T) = \cdots$$

$$\ddot{x}(0) = \cdots, \ddot{x}(T) = \cdots$$

$$\vdots$$

$$p_0 = \cdots,$$

$$p_5 T^5 + p_4 T^4 + p_3 T^3 + p_2 T^2 + p_1 T + p_0 = \cdots$$

$$[T^5, T^4, T^3, T^2, T, 1] \begin{bmatrix} p_5 \\ p_4 \\ p_3 \\ p_2 \\ p_1 \\ p_0 \end{bmatrix} = \cdots$$

最小化Snap的轨迹生成

高阶连续性约束：保证两段之间的4阶导连续

$$f_j^{(k)}(T_j) = f_{j+1}^{(k)}(T_j)$$

$$\Rightarrow \sum_{i \geq k} \frac{i!}{(i-k)!} T_j^{i-k} p_{j,i} - \sum_{l \geq k} \frac{l!}{(l-k)!} T_j^{l-k} p_{j+1,l} = 0$$

$$\Rightarrow \left[\dots \quad \frac{i!}{(i-k)!} T_j^{i-k} \quad \dots \quad - \frac{l!}{(l-k)!} T_j^{l-k} \quad \dots \right] \begin{bmatrix} \vdots \\ p_{j,i} \\ \vdots \\ p_{j+1,l} \\ \vdots \end{bmatrix} = 0$$

$$\Rightarrow [\mathbf{A}_j \quad -\mathbf{A}_{j+1}] \begin{bmatrix} \mathbf{p}_j \\ \mathbf{p}_{j+1} \end{bmatrix} = 0$$

最小化Snap的轨迹生成

线性等式约束的二次规划

$$\begin{aligned} \min \quad & \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} \\ \text{s. t.} \quad & \mathbf{A}_{\text{eq}} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} = \mathbf{d}_{\text{eq}} \end{aligned}$$

最小化Snap的轨迹生成是一个典型的凸优化问题。

解析解法

决策变量映射

- 直接优化多项式轨迹在数值上不稳定
- 更倾向于替代变量，使得优化每段端点的微分
- 我们有 $\mathbf{M}_j \mathbf{p}_j = \mathbf{d}_j$ ， $\mathbf{M}_j$ 是一个映射矩阵，将多项式系数映射至微分

$$J = \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} \quad J = \begin{bmatrix} \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_M \end{bmatrix}^T \begin{bmatrix} \mathbf{M}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{M}_M \end{bmatrix}^{-T} \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{M}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{M}_M \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_M \end{bmatrix}$$

决策变量映射

$$J = \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} \quad + \quad \mathbf{M}_j \mathbf{p}_j = \mathbf{d}_j$$



$$J = \begin{bmatrix} \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_M \end{bmatrix}^T \begin{bmatrix} \mathbf{M}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{M}_M \end{bmatrix}^{-T} \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{M}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{M}_M \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_M \end{bmatrix}$$

$$x(t) = p_5 t^5 + p_4 t^4 + p_3 t^3 + p_2 t^2 + p_1 t + p_0$$

$$x'(t) = 5p_5 t^4 + 4p_4 t^3 + 3p_3 t^2 + 2p_2 t + p_1$$

$$x''(t) = 20p_5 t^3 + 12p_4 t^2 + 6p_3 t + 2p_2$$

$$M = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ T^5 & T^4 & T^3 & T^2 & T & 1 \\ 5T^4 & 4T^3 & 3T^2 & 2T & 1 & 0 \\ 20T^3 & 12T^2 & 6T & 2 & 0 & 0 \end{bmatrix}$$

分离固定变量和自由变量

- 使用选择矩阵 $\mathbf{C}$ 来分离自由变量($\mathbf{d}_P$) 和受约束变量($\mathbf{d}_F$)
 - 自由变量: 微分不受指定, 只受连续性约束

$$\mathbf{C}^T \begin{bmatrix} \mathbf{d}_F \\ \mathbf{d}_P \end{bmatrix} = \begin{bmatrix} \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_M \end{bmatrix} \quad \longrightarrow \quad J = \begin{bmatrix} \mathbf{d}_F \\ \mathbf{d}_P \end{bmatrix}^T \underbrace{\mathbf{C}\mathbf{M}^{-T}\mathbf{Q}\mathbf{M}^{-1}\mathbf{C}^T}_{\mathbf{R}} \begin{bmatrix} \mathbf{d}_F \\ \mathbf{d}_P \end{bmatrix} = \begin{bmatrix} \mathbf{d}_F \\ \mathbf{d}_P \end{bmatrix}^T \begin{bmatrix} \mathbf{R}_{FF} & \mathbf{R}_{FP} \\ \mathbf{R}_{PF} & \mathbf{R}_{PP} \end{bmatrix} \begin{bmatrix} \mathbf{d}_F \\ \mathbf{d}_P \end{bmatrix}$$

- 转变为一个不受约束的二次规划问题, 可直接闭式求解

$$J = \mathbf{d}_F^T \mathbf{R}_{FF} \mathbf{d}_F + \mathbf{d}_F^T \mathbf{R}_{FP} \mathbf{d}_P + \mathbf{d}_P^T \mathbf{R}_{PF} \mathbf{d}_F + \mathbf{d}_P^T \mathbf{R}_{PP} \mathbf{d}_P$$

$$\mathbf{d}_P^* = -\mathbf{R}_{PP}^{-1} \mathbf{R}_{FP}^T \mathbf{d}_F$$

构建选择矩阵

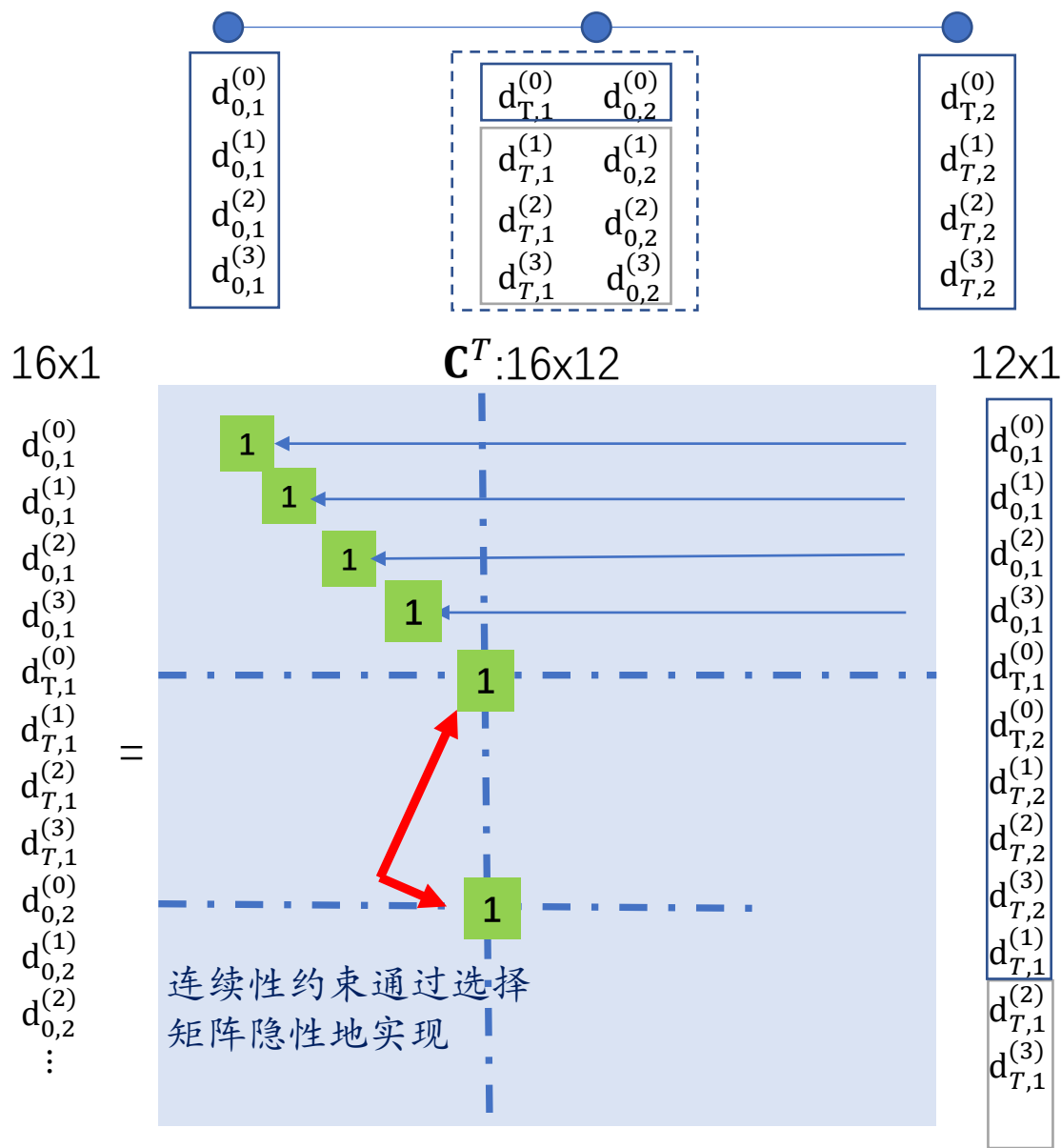
$d_{0,i}^{(k)}$

导数阶数
 段数
 时间

固定的微分项：固定的起点，目标状态以及中间位置

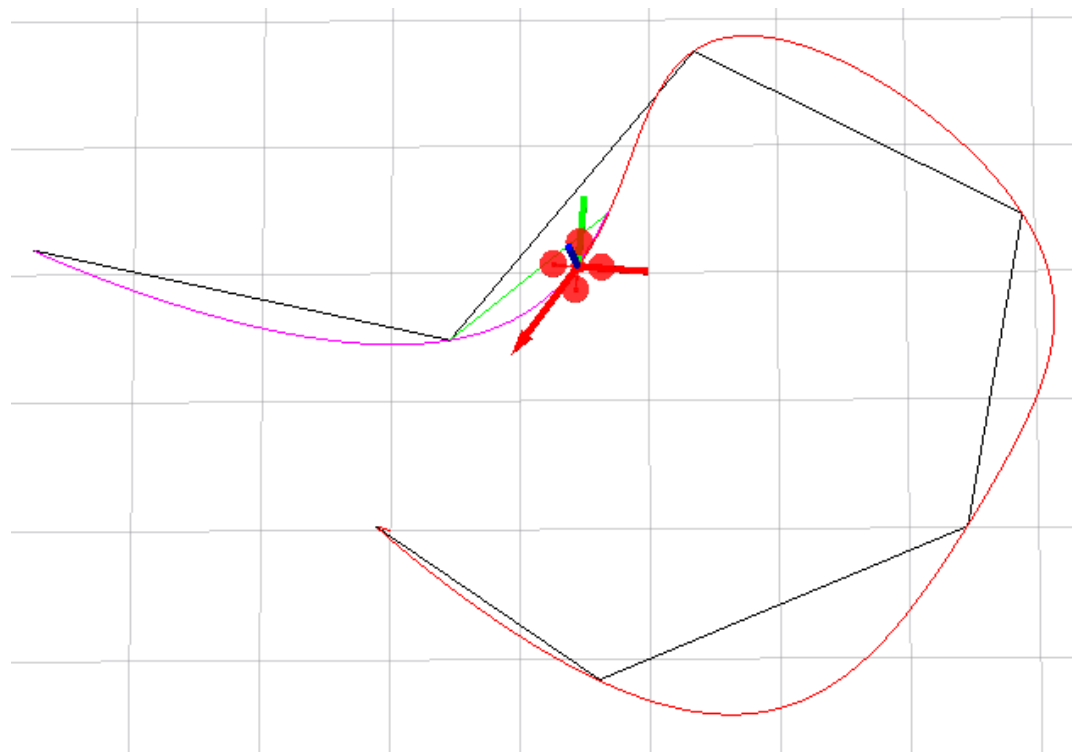
自由的微分项：在中间连接处的所有微分项

$$\begin{bmatrix} \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_M \end{bmatrix} = \mathbf{C}^T \begin{bmatrix} \mathbf{d}_F \\ \mathbf{d}_P \end{bmatrix}$$



和凸优化结果相同

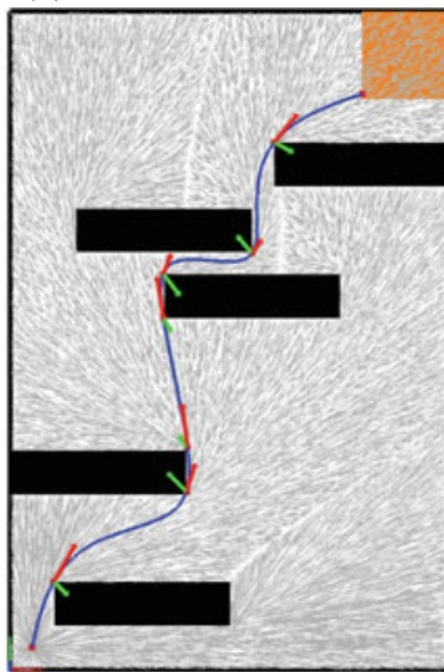
- 最终轨迹



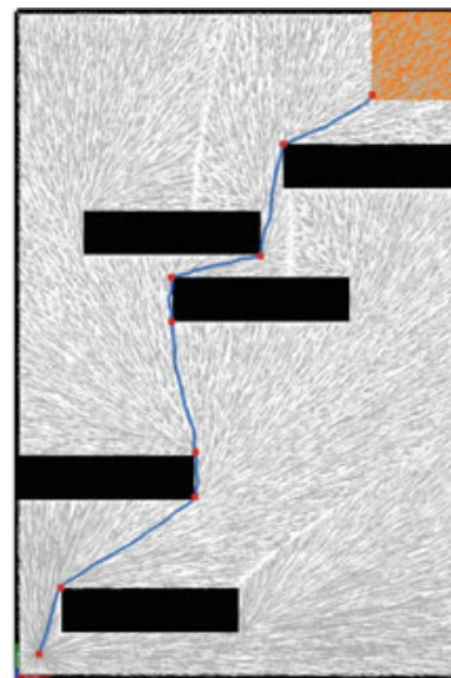
问：怎样获得航点？

层次性规划

- 路径规划+ 轨迹生成
 - 低复杂度：路径规划的问题维度低



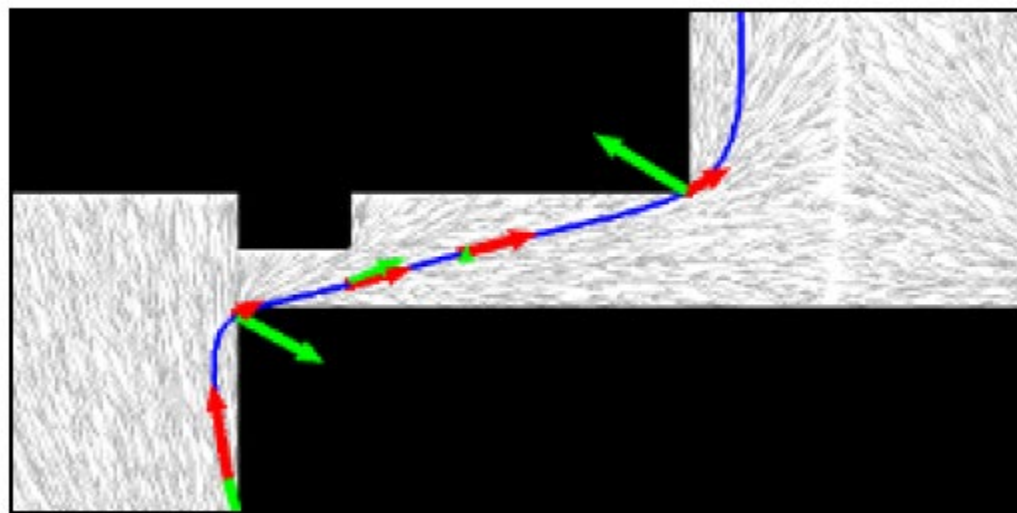
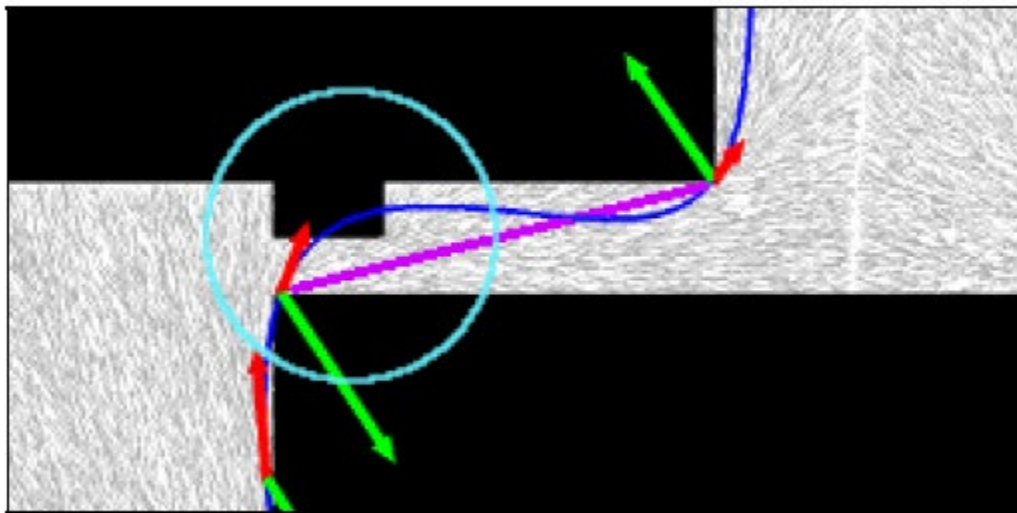
给定航点的轨迹生成具有闭式解



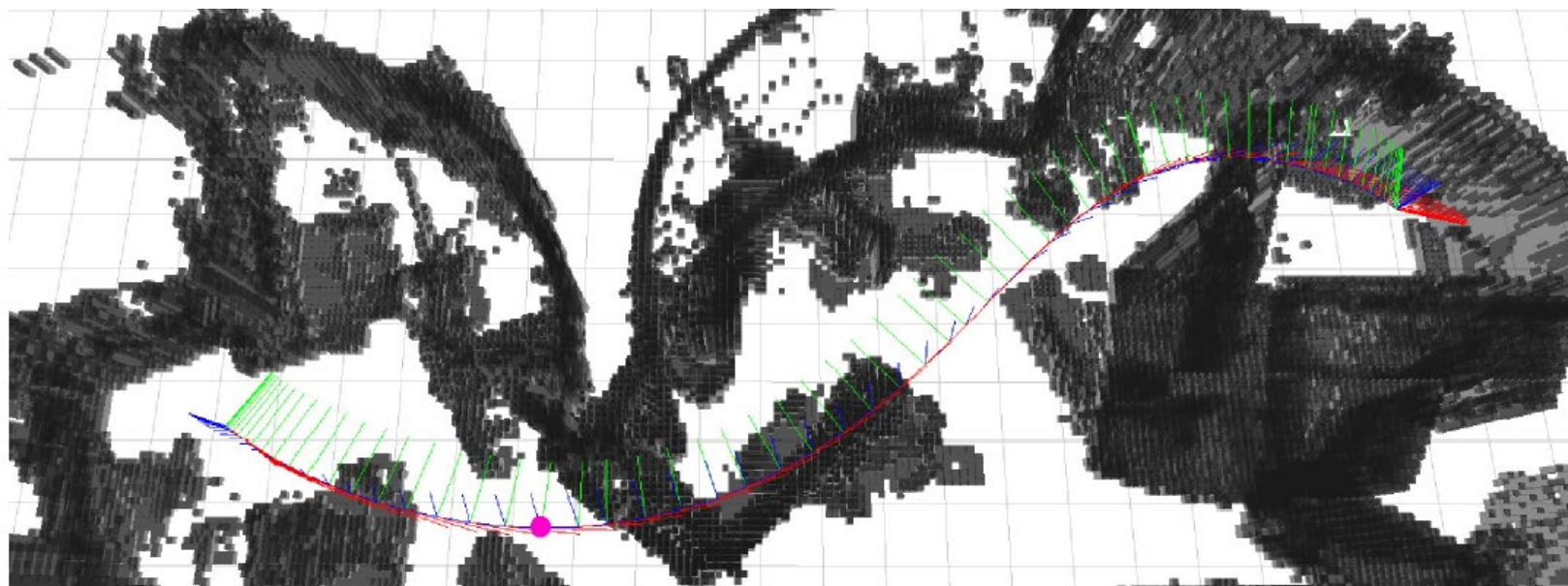
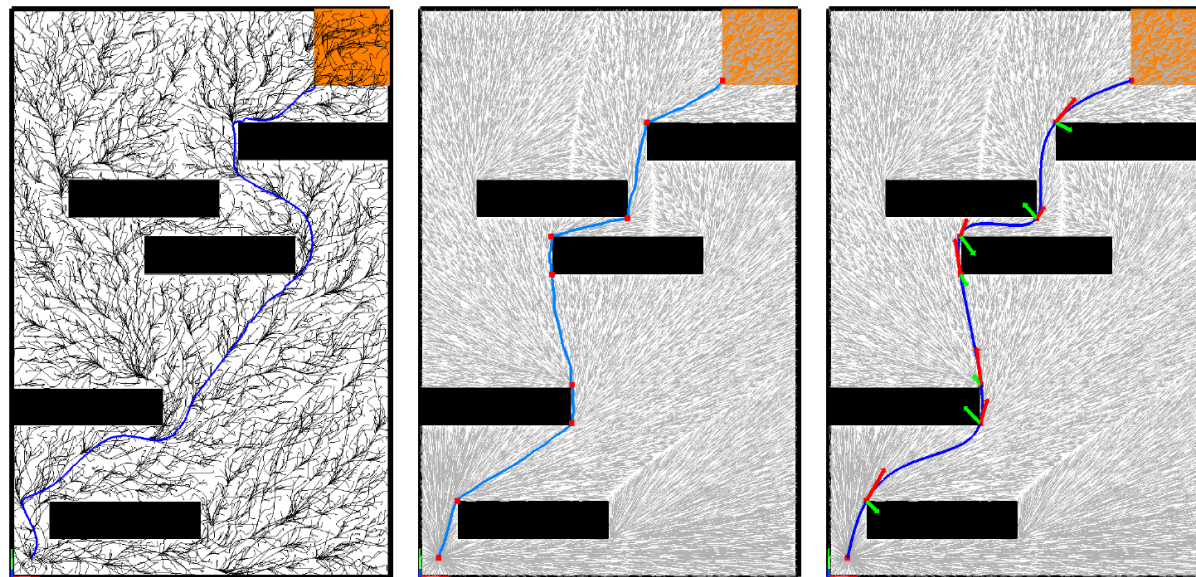
如何获取无碰撞的航点：路径规划

基于闭式解的层次化规划

- 初始路径无碰撞
- 在初始路径的航点上利用闭式解生成轨迹
- 轨迹的碰撞部分则迭代地加入无碰撞的中间航点



RRT*+ 最小化Snap轨迹生成



拓展

归一化

- 时间归一化
 - 很小的时间间隔可能破坏整段轨迹
 - 将很小的时间区间改成合理的数字(1.0).
 - 或者向所有曲线增加比例因子.

$$f(t) = \begin{cases} \sum_{i=0}^N p_{1,i} \left(\frac{t - T_0}{T_1 - T_0} \right)^i & T_0 \leq t \leq T_1 \\ \vdots \\ \sum_{i=0}^N p_{2,i} \left(\frac{t - T_1}{T_2 - T_1} \right)^i & T_1 \leq t \leq T_2 \\ \vdots \\ \sum_{i=0}^N p_{M,i} \left(\frac{t - T_{M-1}}{T_M - T_{M-1}} \right)^i & T_{M-1} \leq t \leq T_M \end{cases}$$

使用相对时间!

- 问题尺度（空间） 归一化
 - 如果问题在大尺度场景下提出.
 - 例如航点的 $x = 100.0\text{ m}$
 - 考虑解决一个小尺度问题（沙盒），然后将结果变换回来

在实践中这两步操作大大提高了数值稳定性

其他工程注意事项

1. 独立还是一起求解3个坐标轴?

$$\min \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{Q}_M \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}$$

$$\text{s.t. } \mathbf{A}_{eq} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix} = \mathbf{d}_{eq}$$

- 通常，求解3个小尺度的问题比1个大尺度的问题更好（稳定，快捷）
- 耦合的轨迹生成可能会在3个坐标轴中产生不同的权重

2. 闭式求解一直更优?

- 当矩阵操作耗费资源，数值求解更鲁棒
- 通用求解器(如Mosek)具有良好的稳定性

3. 多项式可以做任何事?

- 几乎是，但不是所有
- 可以证明多项式方程可以求得最小化单项平方控制输入的最优解。

- 但是如果是
多项式将不再是最优解（超出课程范围）



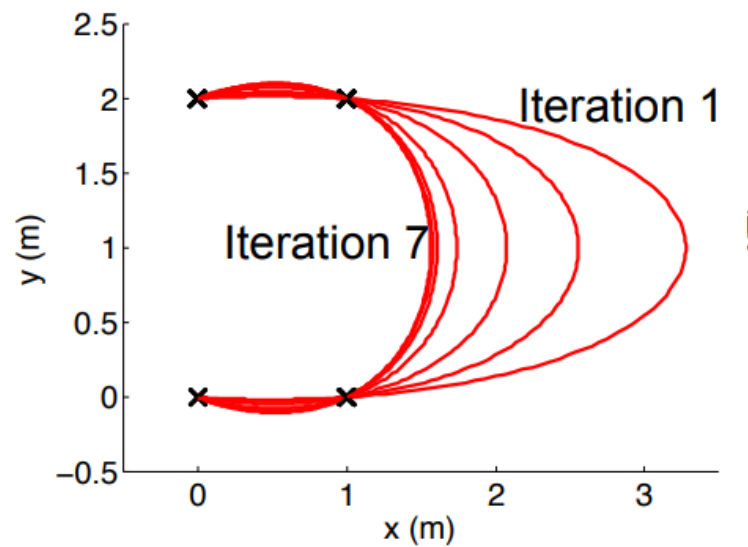
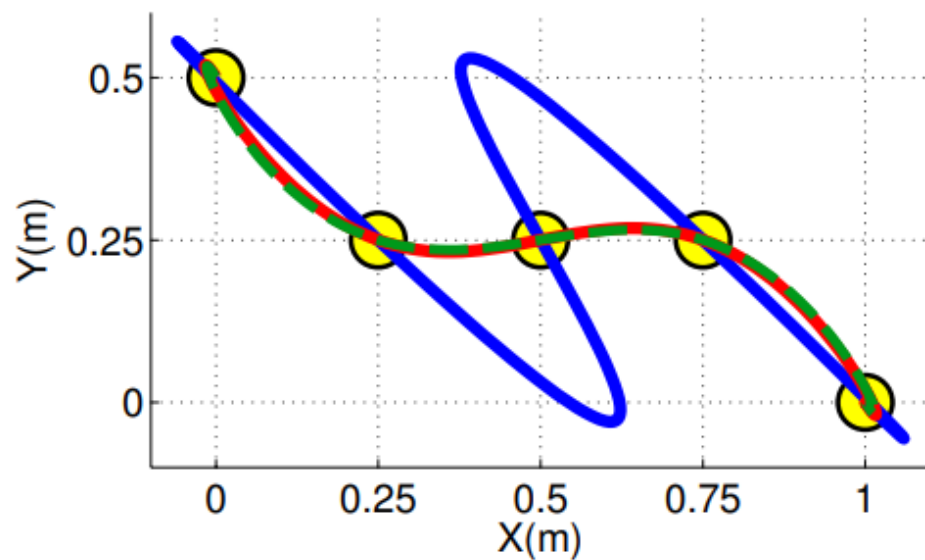
$$J = \int_0^T \rho_1 \cdot jerk^2(t) + \rho_1 \cdot snap^2(t) dt.$$

Efficiency comparison

Benchmark Problem: 3-Segment Joint Optimization	
Method	Solution Time (ms)
MATLAB quadprog.m	9.5
MATLAB Constrained	1.7
MATLAB Unconstrained (Dense)	2.7
C++/Eigen Constrained	0.18
C++/Eigen Unconstrained (Dense)	0.34

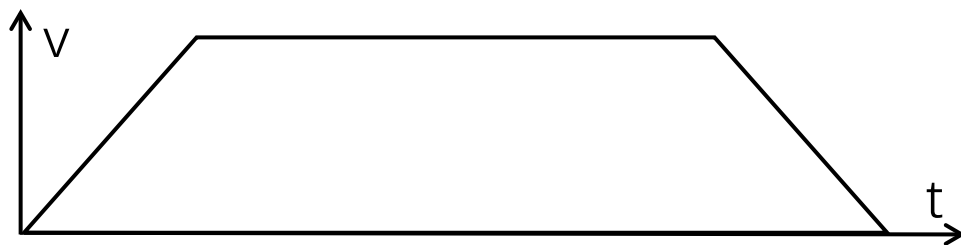
时间分配

- 分段轨迹需要分段时间分配
- 时间分配显著影响最终轨迹
- 如何获得合理的时间分配？



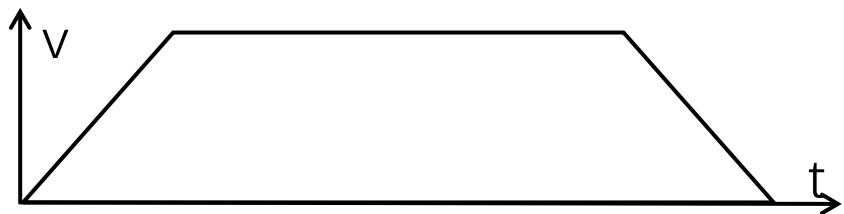
朴素的解决方法

- 使用“梯形速度”时间轮廓来获得每段的时间区间
 - 假设每段轨迹，加速到最大速度→减速到0
 - 加速 + 最大速度 + 减速
- 使用期待的平均速度来获得每段轨迹的时间

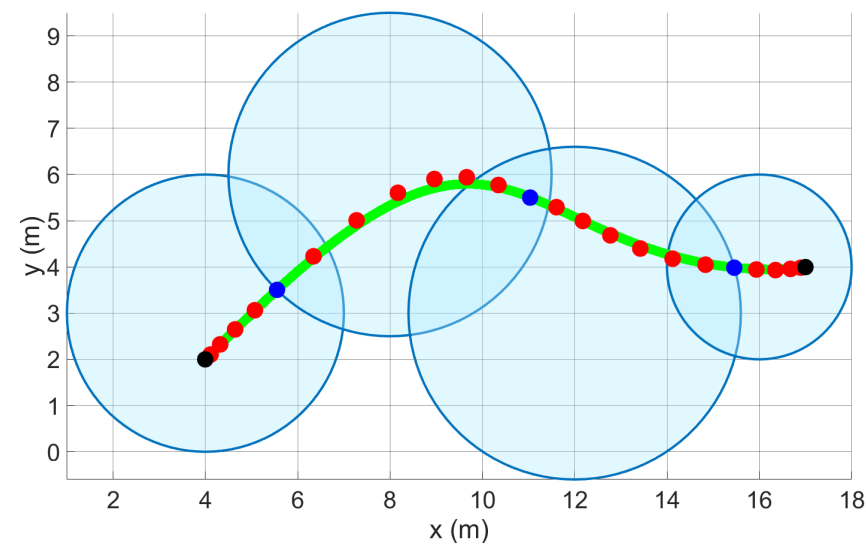
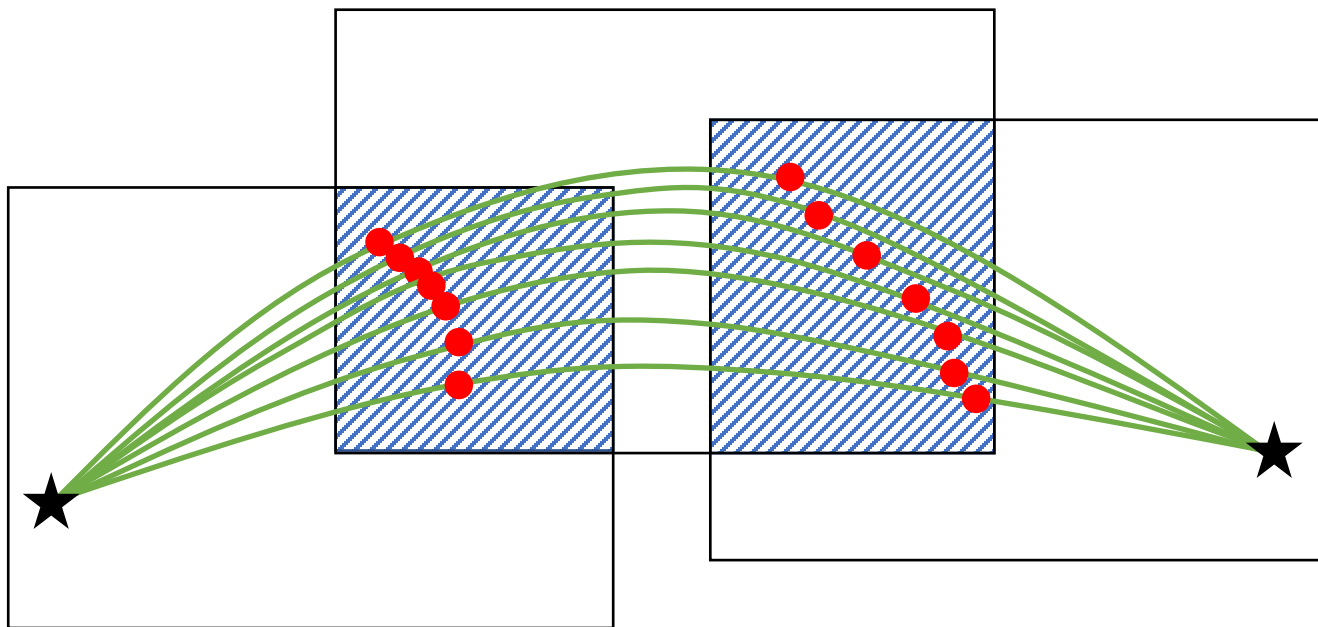


- 远未达到最优
- 只能获得保守的时间轮廓
- 对不同的环境情况没有响应

朴素的解决方法

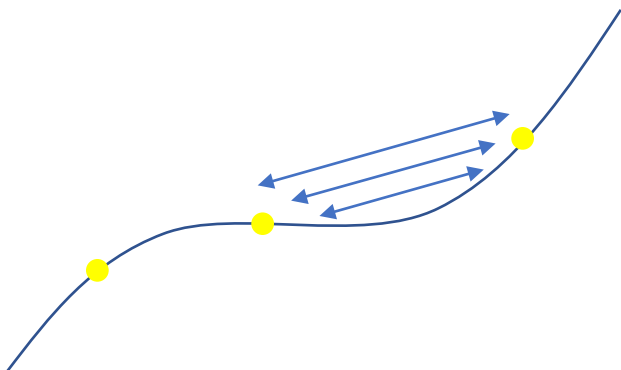


- 作用于一段轨迹非常粗糙
- 在基于飞行走廊的轨迹生成取得不错的效果
- 飞行走廊的重叠区域提供了大片自动调节的空间



直观的解决方法

- 时间区间后处理
 - 假设相对激进的时间分配
 - 生成轨迹
 - 检查每段轨迹的可行性



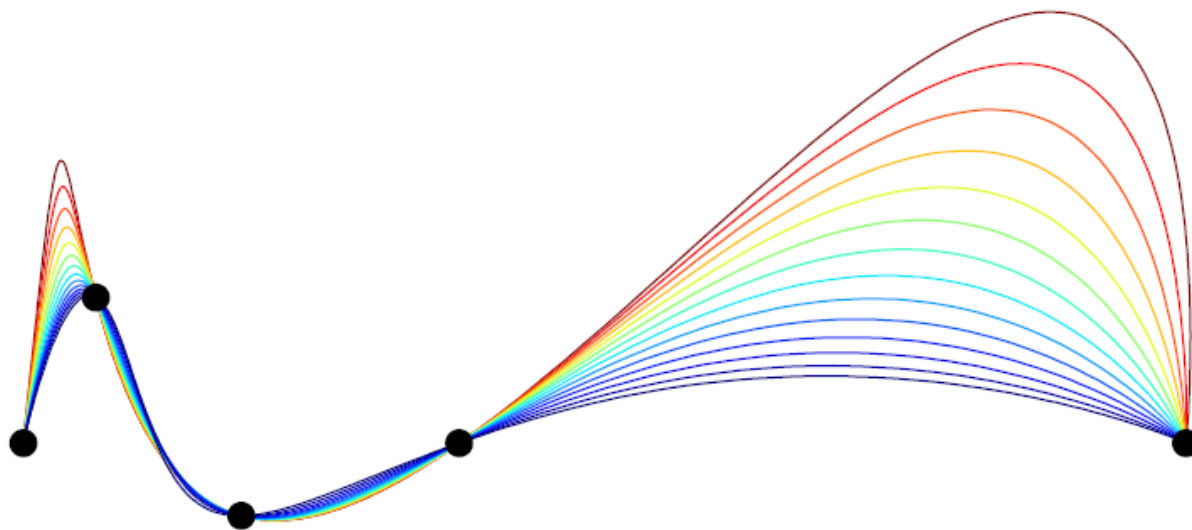
- 增大 T 直到所有可行性条件满足.
- 可能无解

迭代数值求解

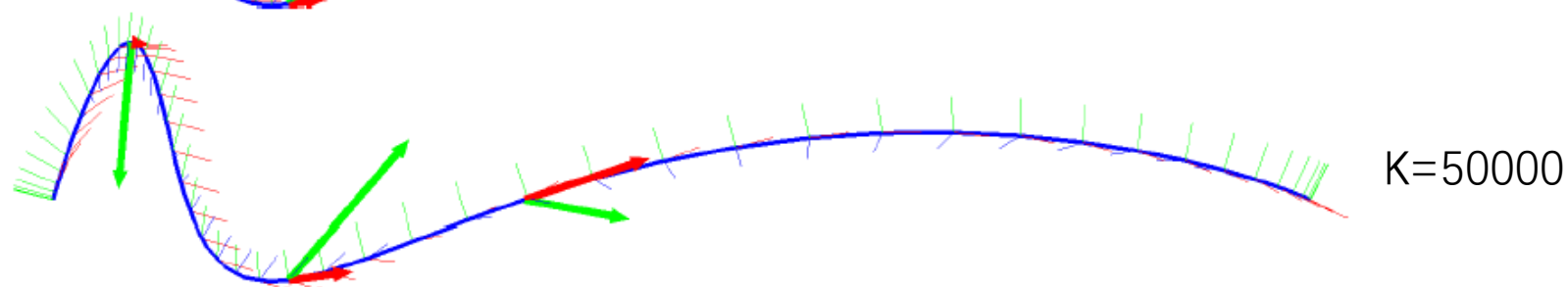
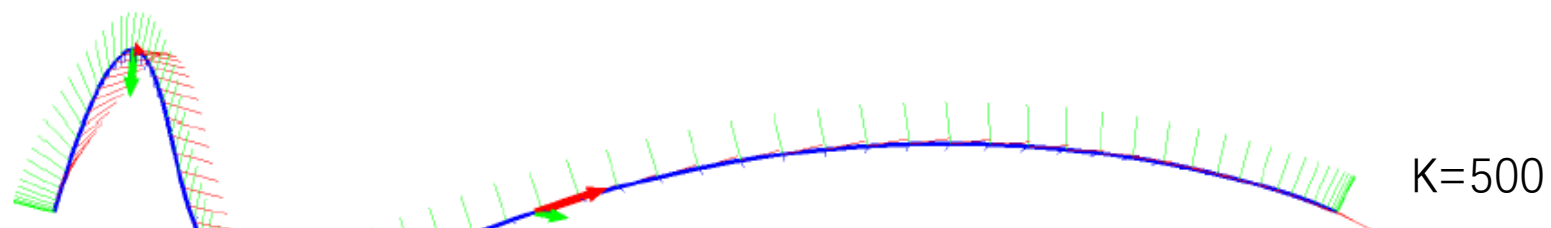
$$J_T = \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T \begin{bmatrix} Q_1(T_1) & & \\ & \ddots & \\ & & Q_M(T_M) \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_M \end{bmatrix}^T + k_T \sum_{i=1}^M T_i$$

在总代价函数中惩罚时间区间

- 最小化目标函数 J
- 数值方法获得 T 的梯度



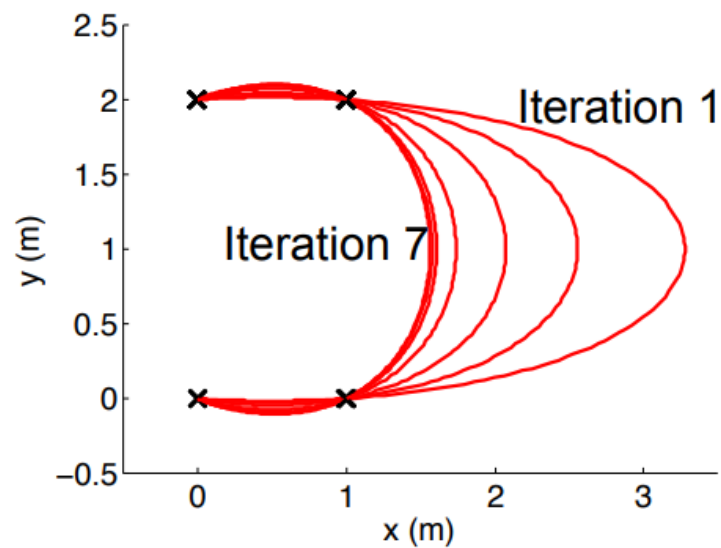
迭代数值求解



- 可调分段时间比率
- 可调总时间

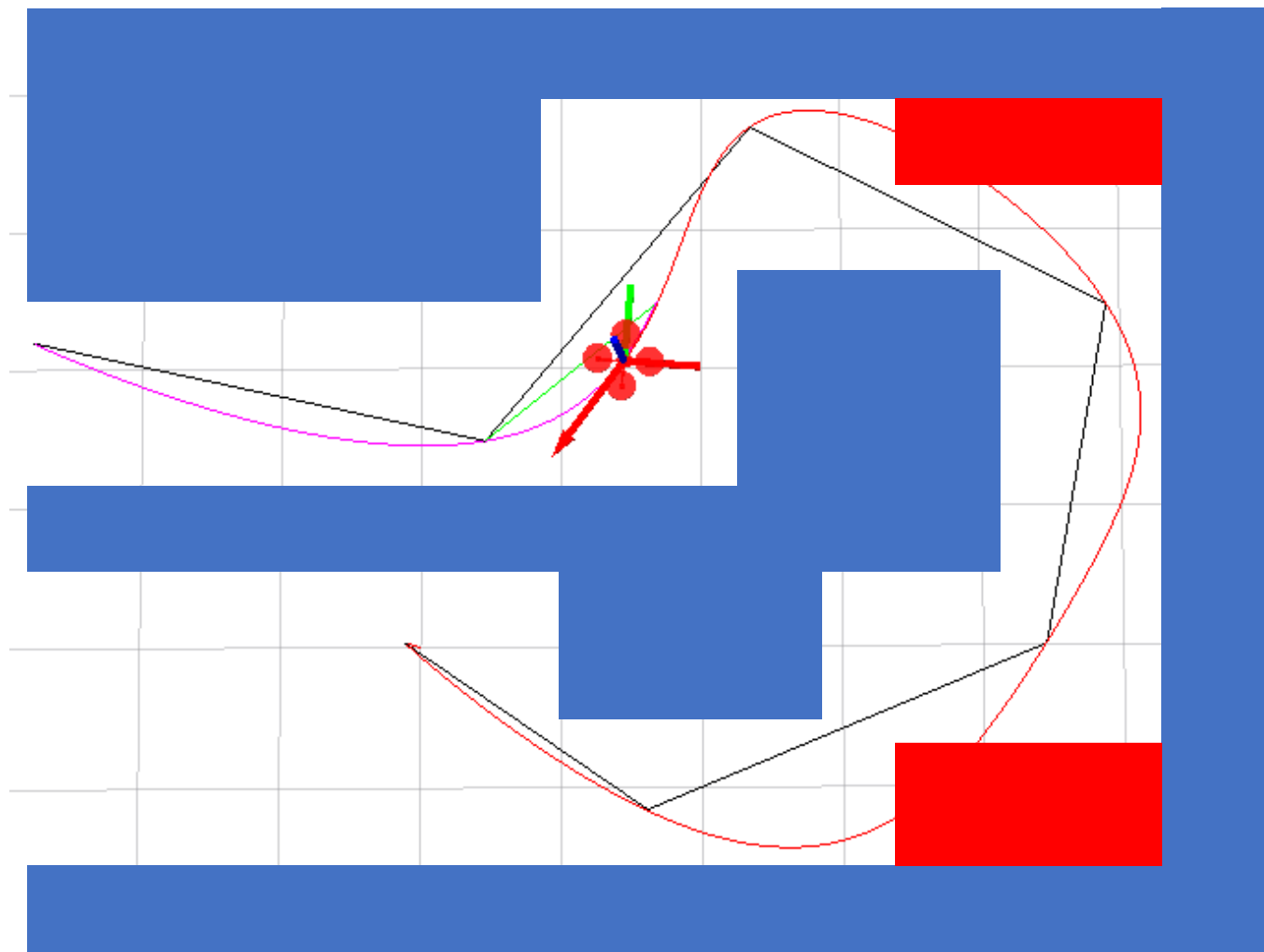
动态约束？

- 无约束，找到最佳时间分配，固定比率
- 保留最佳比率调整总轨迹时间
- 直到约束得到满足



- 可调分段时间比率
- 固定总时间

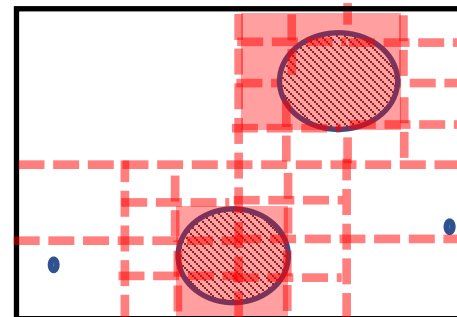
更好的解决办法?



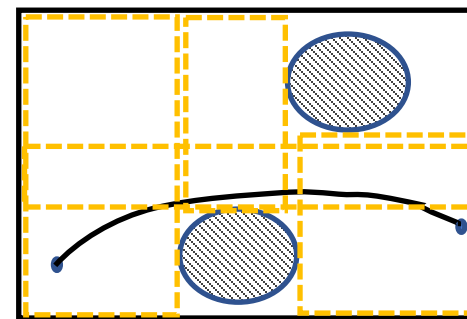
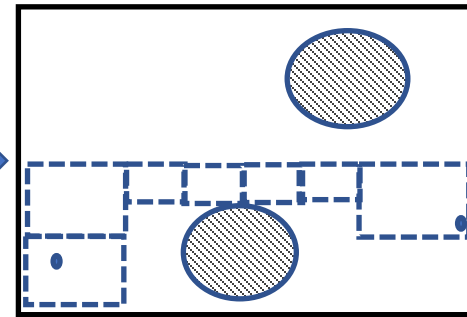
确保避障下生成光滑轨迹



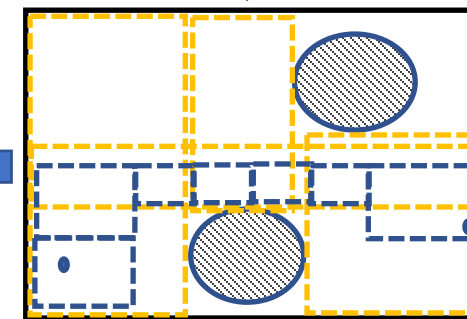
Step 1. 检测障碍物



Step 2. 搜索飞行走廊



Step 4. 生成完全在走廊内的动态可行的轨迹



Step 3. 膨胀飞行走廊

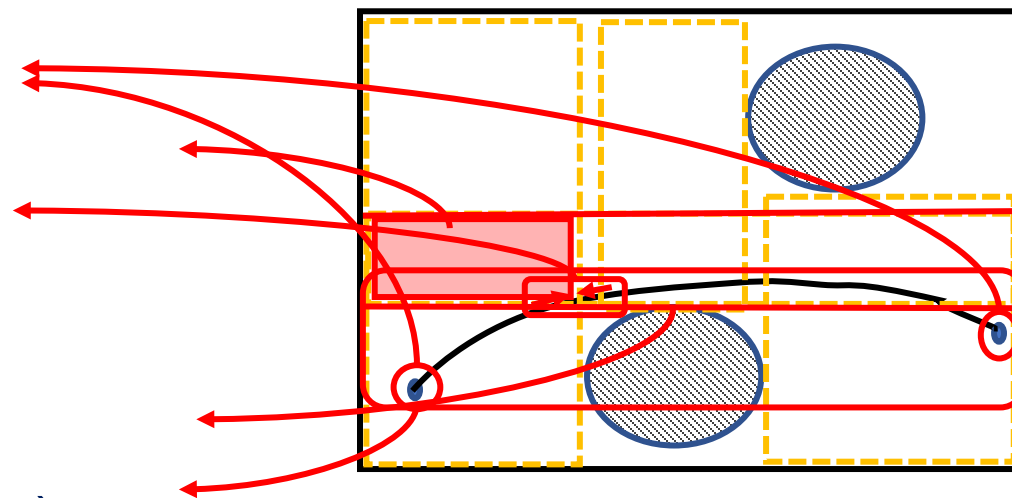
确保避障下生成光滑轨迹：构造约束

- 直接线性约束：

- 起始位置状态约束 ($\mathbf{A}\mathbf{p} = \mathbf{b}$)
- 中间点约束 ($\mathbf{A}\mathbf{p} = \mathbf{b}, \mathbf{A}\mathbf{p} \leq \mathbf{b}$)
- 连续性约束 ($\mathbf{A}\mathbf{p}_i = \mathbf{A}\mathbf{p}_{i+1}$)

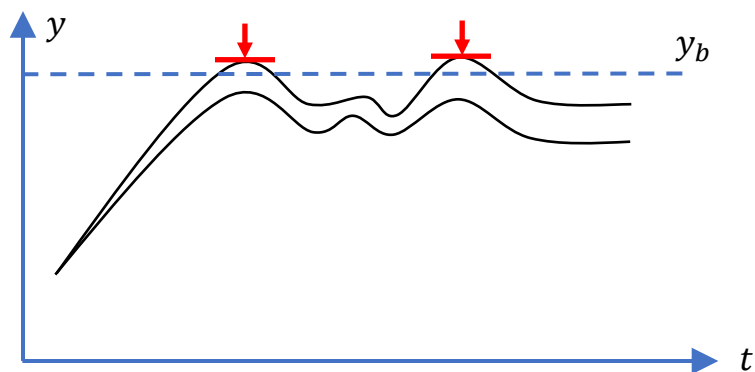
- 中间线性约束：

- 边界条件 ($\mathbf{A}(t)\mathbf{p} \leq \mathbf{b}, \forall t \in [t_l, t_r]$)
- 动态约束 ($\mathbf{A}(t)\mathbf{p} \leq \mathbf{b}, \forall t \in [t_l, t_r]$)
 - 速度约束
 - 加速度约束



约束在全局条件下满足

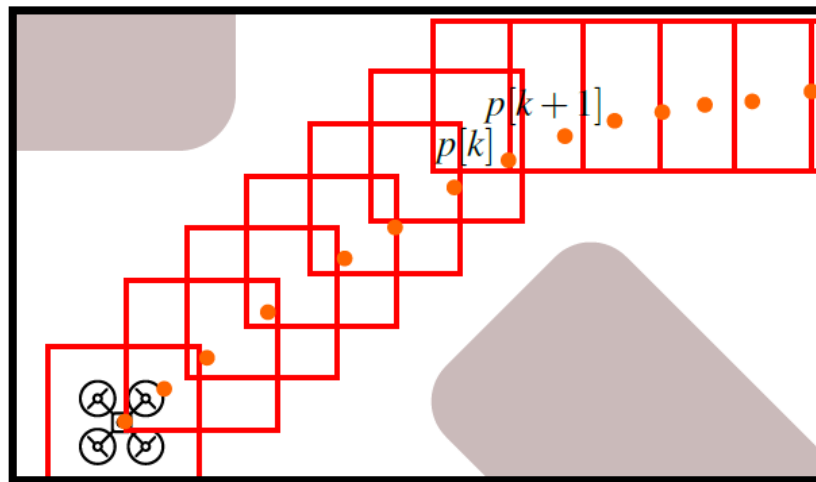
- 迭代检查机制，并增加额外约束



Online generation of collision-free trajectories for quadrotor flight in unknown cluttered environments, J. Chen, **ICRA** 2016

- 迭代求解耗费时间
- 如果没有严格满足所有约束的解。必须迭代10次检查解的存在？

- 在离散时间点上增加数值约束



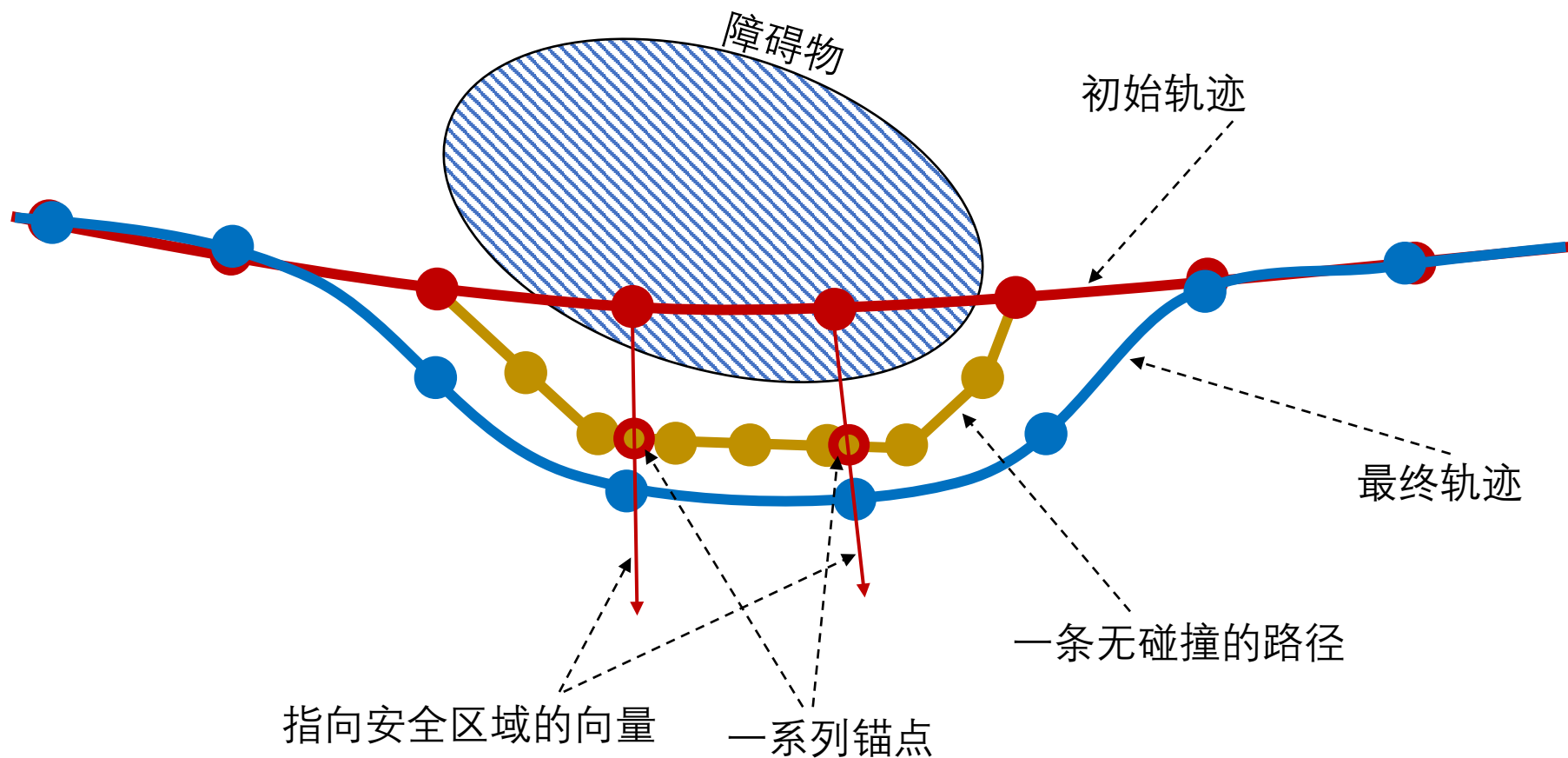
A hybrid method for online trajectory planning of mobile robots in cluttered environments, L Campos-Macías, **RAL** 2017

- 产生过于保守的轨迹
- 太多约束，计算耗费太大

解决方案2：保守约束



解决方案3：软约束





谢谢观看